1. 개요

xv6에서 기본적으로 사용하는 Round Robin 방식의 스케줄러 대신, 티켓 수에 비례한 CPU 시간 분배를 제공하는 결정론적 스케줄러(Stride)를 xv6 커널에 구현했다. 결정론적 스케줄러의 핵심은 같은 조건이면, 항상 같은 선택을 한다는 것이다. 따라서 단기적인 변동성이 존재하고, 예측하기 어려운 Lottery 스케줄러 방식 대비 Stride(결정론적) 스케줄러를 구현해보면서 두 방식의 차이와 장단점을 이해할 수 있었다. 또한 구현 과정에서 커널 스케줄러 교체, proc 구조체 확장, 시스템콜 추가/연결 복습을 하면서 운영체제에 대해 더 이해할 수 있었다.

2. 상세 설계

(1) 수정/추가한 파일 및 함수 정리

[Makefile]

UPROGS에 테스트 프로그램 3개 추가

```
168 UPROGS=\
169     _cat\
170     _echo\
171     _forktest\
172     _grep\
173     _init\
174     _kill\
175     _ln\
176     _ls\
177     _mkdir\
178     _rm\
179     _sh\
180     _stressfs\
181     _usertests\
182     _wc\
183     _zombie\
184     _debug_test\
185     _syscall_test\
186     _scheduler_test
```

[proc.h]

roc 구조체 확장 및 stride 스케줄러에게 필요한 상수 정의

```
37 // Per-process state
38 struct proc {
39   uint sz;                     // Size of process memory (bytes)
40   pde_t* pgdir;                // Page table
41   char *kstack;                // Bottom of kernel stack for this process
42   enum procstate state;        // Process state
43   int pid;                     // Process ID
44   struct proc *parent;         // Parent process
45   struct trapframe *tf;        // Trap frame for current syscall
46   struct context *context;     // swtch() here to run process
47   void *chan;                  // If non-zero, sleeping on chan
48   int killed;                  // If non-zero, have been killed
49   struct file *ofile[NOFILE];  // Open files
50
51   struct inode *cwd;           // Current directory
52   char name[16];               // Process name (debugging)
53   int tickets;
54   uint stride;                 // 기본값: 0(setticket으로 설정)
55   uint pass;                   // 기본값: 0(setticket으로 설정)
56   int ticks;                   // 기본값: 0
57   int end_ticks;               // 기본값: -1(양수인 경우 ticks 변수가 end_ticks 값이 되면 프로세스 종료)
58 };
59
60 #define STRIDE_MAX 100000
61 #define PASS_MAX 15000
62 #define DISTANCE_MAX 7500
63
```

p

[proc.c]

proc 구조체 필드를 초기화하는 코드 추가. 프로세스 구조체 p의 tickets, stride, pass, ticks, end_ticks 등을 초기화한다.

```
75 {
76   struct proc *p;
77   char *sp;
78
79   acquire(&ptable.lock);
80
81   for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
82     if(p->state == UNUSED)
83       goto found;
84
85   release(&ptable.lock);
86   return 0;
87
88 found:
89   p->state = EMBRYO;
90   p->pid = nextpid++;
91
92   release(&ptable.lock);
93
94   // Allocate kernel stack.
95   if((p->kstack = kalloc()) == 0){
96     p->state = UNUSED;
97     return 0;
98   }
99   sp = p->kstack + KSTACKSIZE;
100
101  // Leave room for trap frame.
102  sp -= sizeof *p->tf;
103  p->tf = (struct trapframe*)sp;
104
105  // Set up new context to start executing at forkret,
106  // which returns to trapret.
107  sp -= 4;
108  *(uint*)sp = (uint)trapret;
109
110  sp -= sizeof *p->context;
111  p->context = (struct context*)sp;
112  memset(p->context, 0, sizeof *p->context);
113  p->context->eip = (uint)forkret;
114
115  // stride scheduler 관련 초기화 추가
116  p->tickets   = 1;  // 기본 티켓 수 1
117  p->stride    = 0;  // settickets() 호출 시 계산
118  p->pass      = 0;  // 시작 시 pass = 0
119  p->ticks     = 0;  // 실행된 틱 누적
120  p->end_ticks = -1; // 무제한 실행(end_ticks 설정 전까지)
121
122  return p;
123 }
```

[디버깅 코드 구현]

(1) 커널 모드의 프로세스가 스케줄러에게 CPU 제어권을 넘기기 직전(trap.c의 trap() 내부)

```
 97   // Force process exit if it has been killed and is in user space.
 98   // (If it is still executing in the kernel, let it keep running
 99   // until it gets to the regular system call return.)
100   if(myproc() && myproc()->killed && (tf->cs&3) == DPL_USER)
101     exit();
102
103   // Force process to give up CPU on clock tick.
104   // If interrupts were on while locks held, would need to check nlock.
105   if(myproc() && myproc()->state == RUNNING && tf->trapno == T_IRQ0+IRQ_TIMER){
106           struct proc *p = myproc();
107           // 디버그용 출력
108           if(p->pid > 2 && p->parent && p->parent->pid > 2){
109                   cprintf("Process %d selected, stride : %d, ticket : %d, pass : %d -> %d (%d/%d)\n",
110                                   p->pid, p->stride, p->tickets, p->pass - p->stride, p->pass, p->ticks, p->end_ticks);
111           }
112
113     yield();
114   }
115
116   // Check if the process has been killed since we yielded
117   if(myproc() && myproc()->killed && (tf->cs&3) == DPL_USER)
118     exit();
119 }
```

(2) 프로세스 생성이 완료된 직후(proc.c)

```
222   acquire(&ptable.lock);
223
224   np->state = RUNNABLE;
225
226   if(np->pid > 2 && np->parent->pid > 2){
227           cprintf("Process %d start\n", np->pid);
228   }
229
230   release(&ptable.lock);
231
232   return pid;
233 }
```

(3) 프로세스가 종료되는 시점(proc.c)

```
271   begin_op();
272   iput(curproc->cwd);
273   end_op();
274   curproc->cwd = 0;
275
276   acquire(&ptable.lock);
277
278   // 종료되는 프로세스의 pass를 0으로 초기화
279   curproc->pass = 0;
280
281   // Parent might be sleeping in wait().
282   wakeup1(curproc->parent);
283
284   // Pass abandoned children to init.
285   for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
286     if(p->parent == curproc){
287       p->parent = initproc;
288       if(p->state == ZOMBIE)
289         wakeup1(initproc);
290     }
291   }
292
293   if(curproc->pid > 2 && curproc->parent->pid > 2){
294           cprintf("Process %d exit\n", curproc->pid);
295   }
296
297   // Jump into the scheduler, never to return.
298   curproc->state = ZOMBIE;
299   sched();
300   panic("zombie exit");
301 }
```

[시스템 콜 settickets() 구현]

(1) syscall.h에 시스템콜 번호 추가

```
// System call numbers
#define SYS_fork    1
#define SYS_exit    2
#define SYS_wait    3
#define SYS_pipe    4
#define SYS_read    5
#define SYS_kill    6
#define SYS_exec    7
#define SYS_fstat   8
#define SYS_chdir   9
#define SYS_dup     10
#define SYS_getpid  11
#define SYS_sbrk    12
#define SYS_sleep   13
#define SYS_uptime  14
#define SYS_open    15
#define SYS_write   16
#define SYS_mknod   17
#define SYS_unlink  18
#define SYS_link    19
#define SYS_mkdir   20
#define SYS_close   21
#define SYS_settickets 22
~
~
```

(2) user.h에 시스템콜 프로토타입 추가

```
// system calls
int fork(void);
int exit(void) __attribute__((noreturn));
int wait(void);
int pipe(int*);
int write(int, const void*, int);
int read(int, void*, int);
int close(int);
int kill(int);
int exec(char*, char**);
int open(const char*, int);
int mknod(const char*, short, short);
int unlink(const char*);
int fstat(int fd, struct stat*);
int link(const char*, const char*);
int mkdir(const char*);
int chdir(const char*);
int dup(int);
int getpid(void);
char* sbrk(int);
int sleep(int);
int uptime(void);
int settickets(int tickets, int end_ticks);
```

(3) usys.S 어셈블리에 시스템콜 등록

```
11 SYSCALL(fork)
12 SYSCALL(exit)
13 SYSCALL(wait)
14 SYSCALL(pipe)
15 SYSCALL(read)
16 SYSCALL(write)
17 SYSCALL(close)
18 SYSCALL(kill)
19 SYSCALL(exec)
20 SYSCALL(open)
21 SYSCALL(mknod)
22 SYSCALL(unlink)
23 SYSCALL(fstat)
24 SYSCALL(link)
25 SYSCALL(mkdir)
26 SYSCALL(chdir)
27 SYSCALL(dup)
28 SYSCALL(getpid)
29 SYSCALL(sbrk)
30 SYSCALL(sleep)
31 SYSCALL(uptime)
32 SYSCALL(settickets)
```

(4) syscall.c에 시스템콜 설정

```
100 extern int sys_sbrk(void);
101 extern int sys_sleep(void);
102 extern int sys_unlink(void);
103 extern int sys_wait(void);
104 extern int sys_write(void);
105 extern int sys_uptime(void);
106 extern int sys_settickets(void);
107
108 static int (*syscalls[])(void) = {
109 [SYS_fork]    sys_fork,
110 [SYS_exit]    sys_exit,
111 [SYS_wait]    sys_wait,
112 [SYS_pipe]    sys_pipe,
113 [SYS_read]    sys_read,
114 [SYS_kill]    sys_kill,
115 [SYS_exec]    sys_exec,
116 [SYS_fstat]   sys_fstat,
117 [SYS_chdir]   sys_chdir,
118 [SYS_dup]     sys_dup,
119 [SYS_getpid]  sys_getpid,
120 [SYS_sbrk]    sys_sbrk,
121 [SYS_sleep]   sys_sleep,
122 [SYS_uptime]  sys_uptime,
123 [SYS_open]    sys_open,
124 [SYS_write]   sys_write,
125 [SYS_mknod]   sys_mknod,
126 [SYS_unlink]  sys_unlink,
127 [SYS_link]    sys_link,
128 [SYS_mkdir]   sys_mkdir,
129 [SYS_close]   sys_close,
130 [SYS_settickets] sys_settickets,
131 };
```

(5) sysproc.c에 시스템콜 커널 영역 핸들러 구현

```
// 커널 핸들러(시스템 콜)
// 티켓값이 클수록 stride가 작아지면서 더 자주 선택된다. 따라서 티켓값이 클수록(stride가 작을수록) 더 빨리 프로세스가 종료된다
int
sys_settickets(void)
{
        int tickets, end_ticks;
        struct proc *p = myproc();

        // 사용자 영역에서 tickets, end_ticks에 해당하는 정수 인자 2개를 읽어온다
        if(argint(0, &tickets) < 0)
                return -1;
        if(argint(1, &end_ticks) < 0)
                return -1;

        // 유효성 검사(명세 참고)
        // tickets: 1 이상 && STRIDE_MAX 미만
        if(tickets < 1 || tickets >= STRIDE_MAX)
                return -1;

        // Stride 스케줄러 관련 필드 갱신
        p->tickets = tickets;
        p->stride = STRIDE_MAX / tickets;   // 정수 나눗셈(나머지 버림)

        // 테스트 가이드라인 결과와 같도록, 초기 pass값을 0으로 설정
        p->pass = 0;

        // end_ticks: 1미만이면 무시(기존 값 유지), 1 이상이면 지정
        if(end_ticks >= 1)
                p->end_ticks = end_ticks;

        return 0;
}
```

[Stride 스케줄러 관련 구현]

(1) proc.c에 스케줄러 관련 헬퍼 함수 구현

pick_min_pass_proc(): pass값이 가장 작은 프로세스를 찾아오는 함수

```
// 선형 탐색으로 지금까지 본 pid의 pass들 중에서 가장 값이 작은 프로세스를 선택하는 함수
static struct proc* pick_min_pass_proc(void){
        // pass가 가장 작은 것이 제일 좋은 것이므로 best로 naming
        struct proc *p, *best = 0;
        for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
                // RUNNABLE인 프로세스만 고려한다
                if(p->state != RUNNABLE)
                        continue;

                // pass 최솟값 갖는 프로세스를 선정, 동률이면 pid 작은 것을 선택
                if(best == 0 || p->pass < best->pass || (p->pass == best->pass && p->pid < best->pid)){
                        best = p;
                }
        }
        return best;
}

```

(2) **rebase_passes**: 어떤 프로세스의 pass값이 PASS_MAX 값을 초과하는 경우 pass값을 정규화 해주는 함수. 그 밖에도 각 프로세스 간의 pass값 차이가 크면 distance cutting 등의 기능을 구현했다.

```
// 프로세스들의 pass 값 안정성(숫자 안정성)을 위해 정규화하는 함수
// distance cutting, 정규화의 기능을 한다
static void rebase_passes(void){
        // 0xFFFFFFFF은 <limits.h>의 UINT_MAX와 같다
        // unsigned int의 최대값인데, 이는 아직 최소값을 못 찾은 상태를 표시하는 플래그 역할을 한다
        uint min_pass = 0xFFFFFFFF;
        struct proc *p;

        // 1) RUNNABLE 중 최소 pass 찾기
        for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
                // RUNNABLE인 프로세스만 고려한다
                if(p->state != RUNNABLE)
                        continue;
                if(p->pass < min_pass)
                        min_pass = p->pass;
        }

        if(min_pass == 0xFFFFFFFF)
                return;   //RUNNABLE 없음

        cprintf("Rebase Process Start\n");

        // 2) 모든 RUNNABLE 표준화 + distance cutting
        for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
                if(p->state != RUNNABLE)
                        continue;       // RUNNABLE 아니면 고려하지 않는다

                uint old = p->pass;
                uint newp = old - min_pass;   // 가장 작은 프로세스의 pass만큼 빼기

                // 정규환한 프로세스의 pass값이 DISTANCE_MAX의 값을 넘으면, 해당 프로세스의 pass값을 DISTANCE_MAX값으로 바꿔버린다.(clamp)
                // 이는 모든 프로세스 간의 pass값의 차이가 DISTANCE_MAX 값을 넘기지 않음을 의미한다.
                // 왜냐하면 정규화 후에는 가장 작은 pass값이 0이고, 따라서 어떤 프로세스의 pass값이 DISTANCE_MAX 이상만 아니라면, 모든 프로세스 간의 pass값 차이가 DISTANCE_MAX 이하로 형성되기 때문이다.
                if(newp > DISTANCE_MAX){
                        cprintf("Process %d's pass is standardize from %d, with distance cutting, to %d\n", p->pid, (int)old, (int)DISTANCE_MAX);
                        p->pass = DISTANCE_MAX;
                } else {
                // Distance Cutting 상황이 아니라면, 원래대로 정규화만 진행한다.
                        if(old != newp) {
                                cprintf("Process %d's pass is standardize from %d, to %d\n", p->pid, (int)old, (int)newp);
                        }
                        p->pass = newp;
                }
        }
        cprintf("Rebase Process End\n");
}
```

헬퍼 함수를 scheduler(), sched() 함수에서 쓸 수 있게 하기 위해서, proc.c의 맨 위에 함수 원형을 선언해 두었다.

```
23 // Stride 스케줄러 헬퍼 함수 원형
24 static struct proc* pick_min_pass_proc(void);
25 static void rebase_passes(void);
```

(3) proc.c의 scheduler() 수정을 통해 Stride 스케줄러로 교체

scheduler()는 커널의 메인 스케줄 루프로, 프로세스 선택과 실행권 이양에 집중한다. 함수는 무한 루프를 돌며 sti()로 타이머 인터럽트를 허용한 뒤, ptable.lock을 잡고 RUNNABLE 집합에서 다음에 실행할 프로세스를 고른다. 선택 규칙은 pass 최솟값 우선, pass값이 같다면 pid 최솟값으로, 이를 위해 pick_min_pass_proc() 함수를 사용한다. 만약 RUNNABLE이 없다면 락을 풀고 다음 인터럽트를 기다리며 루프를 계속한다.

실행할 프로세스를 정하면, c->proc에 현재 CPU의 실행 대상을 기록하고 switchuvm(p)로 해당 프로세스의 주소 공간을 장착한 뒤 상태를 RUNNING으로 갱신한다. 이어서 swtch(&c->scheduler, p->context)로 컨텍스트를 프로세스로 전환하면, 이제부터는 프로세스가 실행된다.타이머에 의해 선점되거나, yield()/sleep() 등으로 자발적으로 양보하면 결국 sched()를 통해 스케줄러로 다시 돌아온다. 복귀하면 switchkvm()으로 커널 전용 주소 공간으로 되돌리고, c->proc = 0으로 CPU가 현재 어떤 프로세스도 실행하지 않음을 표시한다.

```
342 scheduler(void)
343 {
344   struct cpu *c = mycpu();
345   c->proc = 0;
346
347   for(;;){
348     // Enable interrupts on this processor.
349     sti();
350
351     // Loop over process table looking for process to run.
352     acquire(&ptable.lock);
353
354     // 1) RUNNABLE 중 최소 pass를 선택(pass 값이 같다면, 최소 pid를 선택)
355     struct proc *p = pick_min_pass_proc();
356
357     if(p == 0){
358             // 실행할 것이 없는 경우
359             release(&ptable.lock);
360             continue;
361     }
362
363     // 2) 선택된 프로세스에게 CPU 1틱 부여
364     c->proc = p;
365     switchuvm(p);
366     p->state = RUNNING;
367
368     // 컨텍스트 스위치: p가 타이머에 선점되거나 yield/블록 되면 돌아온다
369     swtch(&c->scheduler, p->context);
370     switchkvm();
371     c->proc = 0;
372
373     // 3) 1틱 실행했다고 보고 갱신(프로세스가 SLEEPING/ZOMBIE가 되어도 상관 없이)
374     // 선택 1회 = 1틱 가정
375     p->ticks++;
376     p->pass += p->stride;
377
378     // 4) PASS_MAX 초과 시 rebase
379     if(p->pass > PASS_MAX){
380             rebase_passes();
381     }
382
383     release(&ptable.lock);
384   }
385 }
386
```

(4) proc.c의 sched() 수정을 통해 프로세스의 제어권 반납 시 회계(Accounting)하도록 수정:
sched()는 프로세스가 CPU 점유를 마치고 스케줄러로 제어권을 넘기는 공통 초크포인트다. 모든 경로(타이머 선점으로 인한 yield(), 자발 yield(), sleep() 진입, 종료 전환)가 반드시 sched()로 수렴하므로, 여기서 한 번만 회계를 처리하면 누락, 중복 없이 명세를 충족할 수 있다. 구체적으로, sched()는 ptable.lock 보유, 인터럽트 상태 검증 등을 확인한 뒤, 컨텍스트를 스케줄러로 넘기기 직전에 다음을 수행한다. **1.Accounting:** p->ticks++(이번 선택으로 1틱을 사용했다는 사용 내역 기록), p->pass += pass->stride(Stride 규칙에 따라 다음 선택의 공정성을 위한 누적값 갱신), **2. 정규화(필요 시):** if(p->pass > PASS_MAX) rebase_passes();: PASS 오버플로 방지 및 숫자 안정성을 확보한다. 그 후 swtch(&p->context, mycpu()->scheduler)로 실제 제어권을 스케줄러 컨텍스트로 전환한다. 스케줄러는 곧바로 다음 실행 대상을 선택해 동일한 사이클을 반복한다. sched() 내부에 회계 기능을 구현한 이유는 명세 4p에 따라 “각 프로세스가 1틱만큼 cpu 점유를 마치고 제어권을 넘길 때, 본인의 stride 값만큼 pass 값을 올림”이라는 조건을 정확히 구현하기 위해서이다.

```
401 void
402 sched(void)
403 {
404   int intena;
405   struct proc *p = myproc();
406
407   if(!holding(&ptable.lock))
408     panic("sched ptable.lock");
409   if(mycpu()->ncli != 1)
410     panic("sched locks");
411   if(p->state == RUNNING)
412     panic("sched running");
413   if(readeflags()&FL_IF)
414     panic("sched interruptible");
415
416    // 각 프로세스가 1틱만큼 CPU 점유를 마치고 제어권을 넘길 때 본인의 stride 값만큼 pass 값을 올림(명세)
417     p->ticks++;
418     p->pass += p->stride;
419
420     // 프로세스의 pass값이 PASS_MAX 초과 시 rebase_passes() 호출
421     if(p->pass > PASS_MAX){
422             rebase_passes();
423 }
424
425   intena = mycpu()->intena;
426   swtch(&p->context, mycpu()->scheduler);
427   mycpu()->intena = intena;
428 }
```

(2) 구현한 함수 프로토타입

[추가 또는 수정한 함수]

[proc.c]

int settickets(int tickets, int end_tickets); : 현재 프로세스의 tickets/stride/end_ticks 설정용 커널 헬퍼)

static struct proc* pick_min_pass_proc(void); : RUNNABLE 프로세스 중 최소 pass를 갖는 프로세스를 선택하는 함수

static void rebase_passes(void); : RUNNABLE들의 pass 정규화 + distance cutting을 하는 함수

void scheduler(void); : 프로토타입은 기존 scheduler()와 동일, Stride 스케줄러를 위한 로직 추가 -> 1틱 진행되면 ticks++, pass+=stride를 계산하고, 필요 시 rebase_passes()를 진행한다.

static struct proc* allocproc(void); : 프로토타입 동일, 내부에 tickets/stride/pass/ticks/end_ticks 초기화 추가

void exit(void); : 종료되는 프로세스의 pass값을 0으로 초기화하기 위해 curproc->pass=0;을 추가하였다.

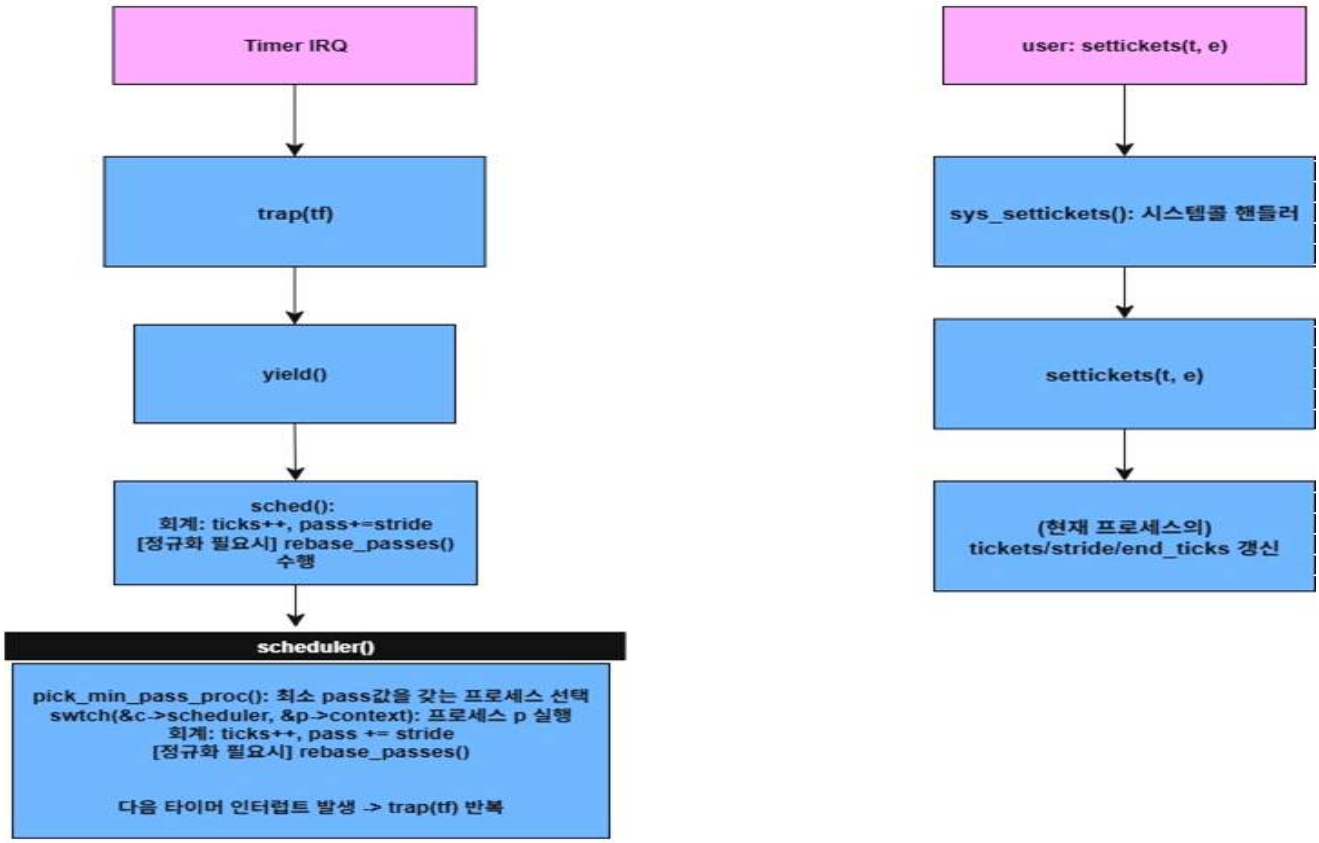
[sysproc.c]

int sys_settickets(void); : 유저 영역에서의 인자를 받아 settickets()를 호출하는 시스템콜 핸들러 함수

[trap.c]

void trap(struct trapframe *tf); : 프로토타입은 기존 trap()과 동일, Stride 스케줄러를 위해 타이머-유저모드 분기에서 로그/종료 등의 로직 추가

(3) 구현한 내용과 기 구현된 함수 간의 호출 그래프



(1) 런타임(타임슬라이스) 사이클 - 핵심 호출 흐름:

Timer IRQ(하드웨어 타이머)가 울리면 커널의 trap(tf)로 들어간다. 유저 모드에서 실행 중인 프로세스라면 타이머 선점을 유도해 한 타임슬라이스를 종료시킬 준비를 한다. trap()은 현재 프로세스가 다음 라운드로 CPU를 양보하도록 yield()를 호출한다. 이때 현재 프로세스의 상태를 RUNNABLE로 바꾸고, 스케줄러로 제어권을 넘기기 위해 sched()를 호출한다. 이때 sched()에서 프로세스가 제어권을 넘기기 직전, 명세에 따라 회계를 한 번만 수행한다.(ticks++, pass+=p->stride), rebase_passes()(필요 시). 회계가 끝나면 swtch(&p->context, myproc()->scheduler)로 스케줄러 컨텍스트로 전환한다. 그 후 scheduler()에서는 선택, 전환을 전담한다. 스케줄러 메인 루프에서 ptable.lock을 잡고 RUNNABLE 집합만 대상으로 pick_Min_pass_proc()을 호출해 pass 최솟값 프로세스를 고른다. 선택되면 switchuvm(p)로 그 프로세스의 주소 공간을 장착하고, p->state = RUNNING으로 바꾼 뒤 swtch(&c->scheduler, p->context)로 실행권을 프로세스에게 이양한다. 선택된 프로세스를 유저 코드로 내려가 실행되다가, 다음 타이머 틱에 다시 위 과정을 통해 제어권을 반납하고 스케줄러로 돌아온다. 이 사이클이 반복되며 Stride의 결정론적 스케줄링이 구현된다.

(2) 시스템콜 경로:

유저의 settickets() 호출은 sys_settickets()를 거쳐 커널의 settickets()로 연결되어 stride, pass, ticks 등을 갱신(초기화)한다
.

3. 결과(다양한 입력에 따른 실행결과 등을 캡쳐 및 해석 등 포함)

- 필수 구현(시스템콜, Stride 스케줄러 등)을 모두 구현한 상태에서의 실행 결과 사진

[$ debug_test 프로그램 실행 결과]

```
$ debug_test
Process 11 start
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (1/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (2/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (3/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (4/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (5/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (6/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (7/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (8/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (9/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (10/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (11/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (12/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (13/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (14/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (15/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (16/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (17/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (18/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (19/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (20/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (21/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (22/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (23/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (24/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (25/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (26/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (27/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (28/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (29/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (30/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (31/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (32/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (33/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (34/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (35/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (36/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (37/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (38/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (39/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (40/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (41/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (42/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (43/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (44/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (45/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (46/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (47/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (48/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (49/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (50/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (51/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (52/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (53/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (54/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (55/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (56/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (57/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (58/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (59/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (60/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (61/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (62/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (63/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (64/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (65/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (66/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (67/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (68/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (69/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (70/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (71/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (72/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (73/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (74/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (75/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (76/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (77/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (78/-1)
Process 11 selected, stride : 0, ticket : 1, pass : 0 -> 0 (79/-1)
result : 1540746712
Process 11 exit
```

debug_test는 출력 형식이 과제 명세를 만족하는지 확인하기 위한 프로그램이다. 따라서 출력에서의 틱 및 다른 변수 값은 신경쓰지 않았고, debug 출력 형식과 동일한지만 확인하였다. 이때 테스트 가이드라인의 debug_test의 총 틱 수보다 더 많은 틱을 채운 후 프로세스가 종료되었는데, 이는 CPU마다 다른 값이므로 올바르다 판단하였다.

[$ syscall_test 프로그램 실행 결과]

```
$ syscall_test
Process 13 start
Process 13 selected, stride : 1000, ticket : 100, pass : 0 -> 1000 (1/6)
Process 13 selected, stride : 1000, ticket : 100, pass : 1000 -> 2000 (2/6)
Process 13 selected, stride : 1000, ticket : 100, pass : 2000 -> 3000 (3/6)
Process 13 selected, stride : 1000, ticket : 100, pass : 3000 -> 4000 (4/6)
Process 13 selected, stride : 1000, ticket : 100, pass : 4000 -> 5000 (5/6)
Process 13 selected, stride : 1000, ticket : 100, pass : 5000 -> 6000 (6/6)
Process 13 exit
```

: pid를 제외한 다른 값은 전부 테스트 가이드라인 사진과 동일하다.

[$ scheduler_test 프로그램 실행 결과]
- 각 테스트 케이스는
[test1 결과]

```
$ scheduler_test
test1 start

Process 15 start
Process 16 start
Process 17 start
Process 15 selected, stride : 100, ticket : 1000, pass : 0 -> 100 (1/20)
Process 16 selected, stride : 100, ticket : 1000, pass : 0 -> 100 (1/20)
Process 17 selected, stride : 100, ticket : 1000, pass : 0 -> 100 (1/20)
Process 15 selected, stride : 100, ticket : 1000, pass : 100 -> 200 (2/20)
Process 16 selected, stride : 100, ticket : 1000, pass : 100 -> 200 (2/20)
Process 17 selected, stride : 100, ticket : 1000, pass : 100 -> 200 (2/20)
Process 15 selected, stride : 100, ticket : 1000, pass : 200 -> 300 (3/20)
Process 16 selected, stride : 100, ticket : 1000, pass : 200 -> 300 (3/20)
Process 17 selected, stride : 100, ticket : 1000, pass : 200 -> 300 (3/20)
Process 15 selected, stride : 100, ticket : 1000, pass : 300 -> 400 (4/20)
Process 16 selected, stride : 100, ticket : 1000, pass : 300 -> 400 (4/20)
Process 17 selected, stride : 100, ticket : 1000, pass : 300 -> 400 (4/20)
Process 15 selected, stride : 100, ticket : 1000, pass : 400 -> 500 (5/20)
Process 16 selected, stride : 100, ticket : 1000, pass : 400 -> 500 (5/20)
Process 17 selected, stride : 100, ticket : 1000, pass : 400 -> 500 (5/20)
Process 15 selected, stride : 100, ticket : 1000, pass : 500 -> 600 (6/20)
Process 16 selected, stride : 100, ticket : 1000, pass : 500 -> 600 (6/20)
Process 17 selected, stride : 100, ticket : 1000, pass : 500 -> 600 (6/20)
Process 15 selected, stride : 100, ticket : 1000, pass : 600 -> 700 (7/20)
Process 16 selected, stride : 100, ticket : 1000, pass : 600 -> 700 (7/20)
Process 17 selected, stride : 100, ticket : 1000, pass : 600 -> 700 (7/20)
Process 15 selected, stride : 100, ticket : 1000, pass : 700 -> 800 (8/20)
Process 16 selected, stride : 100, ticket : 1000, pass : 700 -> 800 (8/20)
Process 17 selected, stride : 100, ticket : 1000, pass : 700 -> 800 (8/20)
Process 15 selected, stride : 100, ticket : 1000, pass : 800 -> 900 (9/20)
Process 16 selected, stride : 100, ticket : 1000, pass : 800 -> 900 (9/20)
Process 17 selected, stride : 100, ticket : 1000, pass : 800 -> 900 (9/20)
Process 15 selected, stride : 100, ticket : 1000, pass : 900 -> 1000 (10/20)
Process 16 selected, stride : 100, ticket : 1000, pass : 900 -> 1000 (10/20)
Process 17 selected, stride : 100, ticket : 1000, pass : 900 -> 1000 (10/20)
Process 15 selected, stride : 100, ticket : 1000, pass : 1000 -> 1100 (11/20)
Process 16 selected, stride : 100, ticket : 1000, pass : 1000 -> 1100 (11/20)
Process 17 selected, stride : 100, ticket : 1000, pass : 1000 -> 1100 (11/20)
Process 15 selected, stride : 100, ticket : 1000, pass : 1100 -> 1200 (12/20)
Process 16 selected, stride : 100, ticket : 1000, pass : 1100 -> 1200 (12/20)
Process 17 selected, stride : 100, ticket : 1000, pass : 1100 -> 1200 (12/20)
Process 15 selected, stride : 100, ticket : 1000, pass : 1200 -> 1300 (13/20)
Process 16 selected, stride : 100, ticket : 1000, pass : 1200 -> 1300 (13/20)
Process 17 selected, stride : 100, ticket : 1000, pass : 1200 -> 1300 (13/20)
Process 15 selected, stride : 100, ticket : 1000, pass : 1300 -> 1400 (14/20)
Process 16 selected, stride : 100, ticket : 1000, pass : 1300 -> 1400 (14/20)
Process 17 selected, stride : 100, ticket : 1000, pass : 1300 -> 1400 (14/20)
Process 15 selected, stride : 100, ticket : 1000, pass : 1400 -> 1500 (15/20)
Process 16 selected, stride : 100, ticket : 1000, pass : 1400 -> 1500 (15/20)
Process 17 selected, stride : 100, ticket : 1000, pass : 1400 -> 1500 (15/20)
Process 15 selected, stride : 100, ticket : 1000, pass : 1500 -> 1600 (16/20)
Process 16 selected, stride : 100, ticket : 1000, pass : 1500 -> 1600 (16/20)
Process 17 selected, stride : 100, ticket : 1000, pass : 1500 -> 1600 (16/20)
Process 15 selected, stride : 100, ticket : 1000, pass : 1600 -> 1700 (17/20)
Process 16 selected, stride : 100, ticket : 1000, pass : 1600 -> 1700 (17/20)
Process 17 selected, stride : 100, ticket : 1000, pass : 1600 -> 1700 (17/20)
Process 15 selected, stride : 100, ticket : 1000, pass : 1700 -> 1800 (18/20)
Process 16 selected, stride : 100, ticket : 1000, pass : 1700 -> 1800 (18/20)
Process 17 selected, stride : 100, ticket : 1000, pass : 1700 -> 1800 (18/20)
Process 15 selected, stride : 100, ticket : 1000, pass : 1800 -> 1900 (19/20)
Process 16 selected, stride : 100, ticket : 1000, pass : 1800 -> 1900 (19/20)
Process 17 selected, stride : 100, ticket : 1000, pass : 1800 -> 1900 (19/20)
Process 15 selected, stride : 100, ticket : 1000, pass : 1900 -> 2000 (20/20)
Process 15 exit
Process 16 selected, stride : 100, ticket : 1000, pass : 1900 -> 2000 (20/20)
Process 16 exit
Process 17 selected, stride : 100, ticket : 1000, pass : 1900 -> 2000 (20/20)
Process 17 exit

test1 end
```

PID 15, 16, 17이며 모두 ticket = 1000으로 같은 티켓을 가지고 있는 경우의 테스트이다.
모두 티켓이 같으므로 stride = STRIDE_MAX / tickets = 100000 / 1000 = 100이다. 따라서 로그에 stride : 100, ticket : 100이 출력된다. 초기 pass는 0이며, end_ticks = 20이 되면 프로세스가 종료된다. 이때 pass가 같으면 더 작은 PID 프로세스가 먼저 스케줄링되므로, PID가 작은 순인 15 - 16 - 17 순으로 프로세스가 종료된다. 즉 동일 티켓을 준 3개 프로세스가 완전히 공정하게 1틱씩 번갈아 실행된다.
[test2 결과]

```
test2 start

Process 18 start
Process 19 start
Process 20 start
Process 18 selected, stride : 1000, ticket : 100, pass : 0 -> 1000 (1/20)
Process 19 selected, stride : 500, ticket : 200, pass : 0 -> 500 (1/20)
Process 20 selected, stride : 333, ticket : 300, pass : 0 -> 333 (1/20)
Process 20 selected, stride : 333, ticket : 300, pass : 333 -> 666 (2/20)
Process 19 selected, stride : 500, ticket : 200, pass : 500 -> 1000 (2/20)
Process 20 selected, stride : 333, ticket : 300, pass : 666 -> 999 (3/20)
Process 20 selected, stride : 333, ticket : 300, pass : 999 -> 1332 (4/20)
Process 18 selected, stride : 1000, ticket : 100, pass : 1000 -> 2000 (2/20)
Process 19 selected, stride : 500, ticket : 200, pass : 1000 -> 1500 (3/20)
Process 20 selected, stride : 333, ticket : 300, pass : 1332 -> 1665 (5/20)
Process 19 selected, stride : 500, ticket : 200, pass : 1500 -> 2000 (4/20)
Process 20 selected, stride : 333, ticket : 300, pass : 1665 -> 1998 (6/20)
Process 20 selected, stride : 333, ticket : 300, pass : 1998 -> 2331 (7/20)
Process 18 selected, stride : 1000, ticket : 100, pass : 2000 -> 3000 (3/20)
Process 19 selected, stride : 500, ticket : 200, pass : 2000 -> 2500 (5/20)
Process 20 selected, stride : 333, ticket : 300, pass : 2331 -> 2664 (8/20)
Process 19 selected, stride : 500, ticket : 200, pass : 2500 -> 3000 (6/20)
Process 20 selected, stride : 333, ticket : 300, pass : 2664 -> 2997 (9/20)
Process 20 selected, stride : 333, ticket : 300, pass : 2997 -> 3330 (10/20)
Process 18 selected, stride : 1000, ticket : 100, pass : 3000 -> 4000 (4/20)
Process 19 selected, stride : 500, ticket : 200, pass : 3000 -> 3500 (7/20)
Process 20 selected, stride : 333, ticket : 300, pass : 3330 -> 3663 (11/20)
Process 19 selected, stride : 500, ticket : 200, pass : 3500 -> 4000 (8/20)
Process 20 selected, stride : 333, ticket : 300, pass : 3663 -> 3996 (12/20)
Process 20 selected, stride : 333, ticket : 300, pass : 3996 -> 4329 (13/20)
Process 18 selected, stride : 1000, ticket : 100, pass : 4000 -> 5000 (5/20)
Process 19 selected, stride : 500, ticket : 200, pass : 4000 -> 4500 (9/20)
Process 20 selected, stride : 333, ticket : 300, pass : 4329 -> 4662 (14/20)
Process 19 selected, stride : 500, ticket : 200, pass : 4500 -> 5000 (10/20)
Process 20 selected, stride : 333, ticket : 300, pass : 4662 -> 4995 (15/20)
Process 20 selected, stride : 333, ticket : 300, pass : 4995 -> 5328 (16/20)
Process 18 selected, stride : 1000, ticket : 100, pass : 5000 -> 6000 (6/20)
Process 19 selected, stride : 500, ticket : 200, pass : 5000 -> 5500 (11/20)
Process 20 selected, stride : 333, ticket : 300, pass : 5328 -> 5661 (17/20)
Process 19 selected, stride : 500, ticket : 200, pass : 5500 -> 6000 (12/20)
Process 20 selected, stride : 333, ticket : 300, pass : 5661 -> 5994 (18/20)
Process 20 selected, stride : 333, ticket : 300, pass : 5994 -> 6327 (19/20)
Process 18 selected, stride : 1000, ticket : 100, pass : 6000 -> 7000 (7/20)
Process 19 selected, stride : 500, ticket : 200, pass : 6000 -> 6500 (13/20)
Process 20 selected, stride : 333, ticket : 300, pass : 6327 -> 6660 (20/20)
Process 20 exit
Process 19 selected, stride : 500, ticket : 200, pass : 6500 -> 7000 (14/20)
Process 18 selected, stride : 1000, ticket : 100, pass : 7000 -> 8000 (8/20)
Process 19 selected, stride : 500, ticket : 200, pass : 7000 -> 7500 (15/20)
Process 19 selected, stride : 500, ticket : 200, pass : 7500 -> 8000 (16/20)
Process 18 selected, stride : 1000, ticket : 100, pass : 8000 -> 9000 (9/20)
Process 19 selected, stride : 500, ticket : 200, pass : 8000 -> 8500 (17/20)
Process 19 selected, stride : 500, ticket : 200, pass : 8500 -> 9000 (18/20)
Process 18 selected, stride : 1000, ticket : 100, pass : 9000 -> 10000 (10/20)
Process 19 selected, stride : 500, ticket : 200, pass : 9000 -> 9500 (19/20)
Process 19 selected, stride : 500, ticket : 200, pass : 9500 -> 10000 (20/20)
Process 19 exit
Process 18 selected, stride : 1000, ticket : 100, pass : 10000 -> 11000 (11/20)
Process 18 selected, stride : 1000, ticket : 100, pass : 11000 -> 12000 (12/20)
Process 18 selected, stride : 1000, ticket : 100, pass : 12000 -> 13000 (13/20)
Process 18 selected, stride : 1000, ticket : 100, pass : 13000 -> 14000 (14/20)
Process 18 selected, stride : 1000, ticket : 100, pass : 14000 -> 15000 (15/20)
Process 18 selected, stride : 1000, ticket : 100, pass : 15000 -> 16000 (16/20)
Rebase Process Start
Process 18's pass is standardize from 16000, to 0
Rebase Process End
Process 18 selected, stride : 1000, ticket : 100, pass : 0 -> 1000 (17/20)
Process 18 selected, stride : 1000, ticket : 100, pass : 1000 -> 2000 (18/20)
Process 18 selected, stride : 1000, ticket : 100, pass : 2000 -> 3000 (19/20)
Process 18 selected, stride : 1000, ticket : 100, pass : 3000 -> 4000 (20/20)
Process 18 exit

test2 end
```

테스트 2는 PID 18, 19, 20인 프로세스가 각각 서로 다른 ticket을 가지는 경우이다. 이때 ticket이 클수록 stride는 작아지고, stride가 작을수록 더 많이 스케줄링된다. 따라서 ticket이 큰 프로세스일수록 더 빨리 종료된다. 그 결과 PID 20 - 19 - 18 순으로 스케줄링되어 프로세스가 종료되었다. 참고로 각 프로세스의 ticket값은 pid 20: 300, pid 19: 200, pid 18: 100이다.
그리고 중간에 18번 프로세스의 pass값이 16000으로, 이는 STRIDE_MAX값인 15000보다 큰 값이다. 그래서 rebase되어 pass가

0으로 줄어드는 것을 관찰할 수 있다.

[test3 결과]

```
test3 start

Process 21 start
Process 22 start
Process 23 start
Process 24 start
Process 25 start
Process 26 start
Process 21 selected, stride : 285, ticket : 350, pass : 0 -> 285 (1/10)
Process 22 selected, stride : 800, ticket : 125, pass : 0 -> 800 (1/10)
Process 23 selected, stride : 111, ticket : 900, pass : 0 -> 111 (1/10)
Process 24 selected, stride : 137, ticket : 725, pass : 0 -> 137 (1/10)
Process 25 selected, stride : 190, ticket : 525, pass : 0 -> 190 (1/10)
Process 26 selected, stride : 125, ticket : 800, pass : 0 -> 125 (1/10)
Process 23 selected, stride : 111, ticket : 900, pass : 111 -> 222 (2/10)
Process 26 selected, stride : 125, ticket : 800, pass : 125 -> 250 (2/10)
Process 24 selected, stride : 137, ticket : 725, pass : 137 -> 274 (2/10)
Process 25 selected, stride : 190, ticket : 525, pass : 190 -> 380 (2/10)
Process 23 selected, stride : 111, ticket : 900, pass : 222 -> 333 (3/10)
Process 26 selected, stride : 125, ticket : 800, pass : 250 -> 375 (3/10)
Process 24 selected, stride : 137, ticket : 725, pass : 274 -> 411 (3/10)
Process 21 selected, stride : 285, ticket : 350, pass : 285 -> 570 (2/10)
Process 23 selected, stride : 111, ticket : 900, pass : 333 -> 444 (4/10)
Process 26 selected, stride : 125, ticket : 800, pass : 375 -> 500 (4/10)
Process 25 selected, stride : 190, ticket : 525, pass : 380 -> 570 (3/10)
Process 24 selected, stride : 137, ticket : 725, pass : 411 -> 548 (4/10)
Process 23 selected, stride : 111, ticket : 900, pass : 444 -> 555 (5/10)
Process 26 selected, stride : 125, ticket : 800, pass : 500 -> 625 (5/10)
Process 24 selected, stride : 137, ticket : 725, pass : 548 -> 685 (5/10)
Process 23 selected, stride : 111, ticket : 900, pass : 555 -> 666 (6/10)
Process 21 selected, stride : 285, ticket : 350, pass : 570 -> 855 (3/10)
Process 25 selected, stride : 190, ticket : 525, pass : 570 -> 760 (4/10)
Process 26 selected, stride : 125, ticket : 800, pass : 625 -> 750 (6/10)
Process 23 selected, stride : 111, ticket : 900, pass : 666 -> 777 (7/10)
Process 24 selected, stride : 137, ticket : 725, pass : 685 -> 822 (6/10)
Process 26 selected, stride : 125, ticket : 800, pass : 750 -> 875 (7/10)
Process 25 selected, stride : 190, ticket : 525, pass : 760 -> 950 (5/10)
Process 23 selected, stride : 111, ticket : 900, pass : 777 -> 888 (8/10)
Process 22 selected, stride : 800, ticket : 125, pass : 800 -> 1600 (2/10)
Process 24 selected, stride : 137, ticket : 725, pass : 822 -> 959 (7/10)
Process 21 selected, stride : 285, ticket : 350, pass : 855 -> 1140 (4/10)
Process 26 selected, stride : 125, ticket : 800, pass : 875 -> 1000 (8/10)
Process 23 selected, stride : 111, ticket : 900, pass : 888 -> 999 (9/10)
Process 25 selected, stride : 190, ticket : 525, pass : 950 -> 1140 (6/10)
Process 24 selected, stride : 137, ticket : 725, pass : 959 -> 1096 (8/10)
Process 23 selected, stride : 111, ticket : 900, pass : 999 -> 1110 (10/10)
Process 23 exit
Process 26 selected, stride : 125, ticket : 800, pass : 1000 -> 1125 (9/10)
Process 24 selected, stride : 137, ticket : 725, pass : 1096 -> 1233 (9/10)
Process 26 selected, stride : 125, ticket : 800, pass : 1125 -> 1250 (10/10)
Process 26 exit
Process 21 selected, stride : 285, ticket : 350, pass : 1140 -> 1425 (5/10)
Process 25 selected, stride : 190, ticket : 525, pass : 1140 -> 1330 (7/10)
Process 24 selected, stride : 137, ticket : 725, pass : 1233 -> 1370 (10/10)
Process 24 exit
Process 25 selected, stride : 190, ticket : 525, pass : 1330 -> 1520 (8/10)
Process 21 selected, stride : 285, ticket : 350, pass : 1425 -> 1710 (6/10)
Process 25 selected, stride : 190, ticket : 525, pass : 1520 -> 1710 (9/10)
Process 22 selected, stride : 800, ticket : 125, pass : 1600 -> 2400 (3/10)
Process 21 selected, stride : 285, ticket : 350, pass : 1710 -> 1995 (7/10)
Process 25 selected, stride : 190, ticket : 525, pass : 1710 -> 1900 (10/10)
Process 25 exit
Process 21 selected, stride : 285, ticket : 350, pass : 1995 -> 2280 (8/10)
Process 21 selected, stride : 285, ticket : 350, pass : 2280 -> 2565 (9/10)
Process 22 selected, stride : 800, ticket : 125, pass : 2400 -> 3200 (4/10)
Process 21 selected, stride : 285, ticket : 350, pass : 2565 -> 2850 (10/10)
Process 21 exit
Process 22 selected, stride : 800, ticket : 125, pass : 3200 -> 4000 (5/10)
Process 22 selected, stride : 800, ticket : 125, pass : 4000 -> 4800 (6/10)
Process 22 selected, stride : 800, ticket : 125, pass : 4800 -> 5600 (7/10)
Process 22 selected, stride : 800, ticket : 125, pass : 5600 -> 6400 (8/10)
Process 22 selected, stride : 800, ticket : 125, pass : 6400 -> 7200 (9/10)
Process 22 selected, stride : 800, ticket : 125, pass : 7200 -> 8000 (10/10)
Process 22 exit

test3 end
```

test3는 서로 ticket값이 다른 6개의 프로세스를 Stride 스케줄링 하는 것을 테스트하는 프로그램이다. test2와 비슷한 양상으로, 티켓이 큰 순서대로 더 자주 선택되어 먼저 10틱을 채우고 종료된다.

[test4 결과]

```
test4 start

Process 27 start
Process 28 start
Process 29 start
Process 27 selected, stride : 9090, ticket : 11, pass : 0 -> 9090 (1/20)
Process 28 selected, stride : 4347, ticket : 23, pass : 0 -> 4347 (1/20)
Process 29 selected, stride : 3571, ticket : 28, pass : 0 -> 3571 (1/20)
Process 29 selected, stride : 3571, ticket : 28, pass : 3571 -> 7142 (2/20)
Process 28 selected, stride : 4347, ticket : 23, pass : 4347 -> 8694 (2/20)
Process 29 selected, stride : 3571, ticket : 28, pass : 7142 -> 10713 (3/20)
Process 28 selected, stride : 4347, ticket : 23, pass : 8694 -> 13041 (3/20)
Process 27 selected, stride : 9090, ticket : 11, pass : 9090 -> 18180 (2/20)
Rebase Process Start
Process 27's pass is standardize from 18180, to 7467
Process 28's pass is standardize from 13041, to 2328
Process 29's pass is standardize from 10713, to 0
Rebase Process End
Process 29 selected, stride : 3571, ticket : 28, pass : 0 -> 3571 (4/20)
Process 28 selected, stride : 4347, ticket : 23, pass : 2328 -> 6675 (4/20)
Process 29 selected, stride : 3571, ticket : 28, pass : 3571 -> 7142 (5/20)
Process 28 selected, stride : 4347, ticket : 23, pass : 6675 -> 11022 (5/20)
Process 29 selected, stride : 3571, ticket : 28, pass : 7142 -> 10713 (6/20)
Process 27 selected, stride : 9090, ticket : 11, pass : 7467 -> 16557 (3/20)
Rebase Process Start
Process 27's pass is standardize from 16557, to 5844
Process 28's pass is standardize from 11022, to 309
Process 29's pass is standardize from 10713, to 0
Rebase Process End
Process 29 selected, stride : 3571, ticket : 28, pass : 0 -> 3571 (7/20)
Process 28 selected, stride : 4347, ticket : 23, pass : 309 -> 4656 (6/20)
Process 29 selected, stride : 3571, ticket : 28, pass : 3571 -> 7142 (8/20)
Process 28 selected, stride : 4347, ticket : 23, pass : 4656 -> 9003 (7/20)
Process 27 selected, stride : 9090, ticket : 11, pass : 5844 -> 14934 (4/20)
Process 29 selected, stride : 3571, ticket : 28, pass : 7142 -> 10713 (9/20)
Process 28 selected, stride : 4347, ticket : 23, pass : 9003 -> 13350 (8/20)
Process 29 selected, stride : 3571, ticket : 28, pass : 10713 -> 14284 (10/20)
Process 28 selected, stride : 4347, ticket : 23, pass : 13350 -> 17697 (9/20)
Rebase Process Start
Rebase Process Start
Process 27's pass is standardize from 14934, to 650
Process 28's pass is standardize from 17697, to 3413
Process 29's pass is standardize from 14284, to 0
Rebase Process End
Process 29 selected, stride : 3571, ticket : 28, pass : 0 -> 3571 (11/20)
Process 27 selected, stride : 9090, ticket : 11, pass : 650 -> 9740 (5/20)
Process 28 selected, stride : 4347, ticket : 23, pass : 3413 -> 7760 (10/20)
Process 29 selected, stride : 3571, ticket : 28, pass : 3571 -> 7142 (12/20)
Process 29 selected, stride : 3571, ticket : 28, pass : 7142 -> 10713 (13/20)
Process 28 selected, stride : 4347, ticket : 23, pass : 7760 -> 12107 (11/20)
Process 27 selected, stride : 9090, ticket : 11, pass : 9740 -> 18830 (6/20)
Rebase Process Start
Process 27's pass is standardize from 18830, with distance cutting, to 7500
Process 28's pass is standardize from 12107, to 1394
Process 29's pass is standardize from 10713, to 0
Rebase Process End
Process 29 selected, stride : 3571, ticket : 28, pass : 0 -> 3571 (14/20)
Process 28 selected, stride : 4347, ticket : 23, pass : 1394 -> 5741 (12/20)
Process 29 selected, stride : 3571, ticket : 28, pass : 3571 -> 7142 (15/20)
Process 28 selected, stride : 4347, ticket : 23, pass : 5741 -> 10088 (13/20)
Process 29 selected, stride : 3571, ticket : 28, pass : 7142 -> 10713 (16/20)
Process 27 selected, stride : 9090, ticket : 11, pass : 7500 -> 16590 (7/20)
Rebase Process Start
Process 27's pass is standardize from 16590, to 6502
Process 28's pass is standardize from 10088, to 0
Process 29's pass is standardize from 10713, to 625
Rebase Process End
Process 28 selected, stride : 4347, ticket : 23, pass : 0 -> 4347 (14/20)
Process 29 selected, stride : 3571, ticket : 28, pass : 625 -> 4196 (17/20)
Process 29 selected, stride : 3571, ticket : 28, pass : 4196 -> 7767 (18/20)
Process 28 selected, stride : 4347, ticket : 23, pass : 4347 -> 8694 (15/20)
Process 27 selected, stride : 9090, ticket : 11, pass : 6502 -> 15592 (8/20)
Rebase Process Start
Process 27's pass is standardize from 15592, with distance cutting, to 7500
Process 28's pass is standardize from 8694, to 927
Process 29's pass is standardize from 7767, to 0
Rebase Process End
Process 29 selected, stride : 3571, ticket : 28, pass : 0 -> 3571 (19/20)
Process 28 selected, stride : 4347, ticket : 23, pass : 927 -> 5274 (16/20)
```

```
Process 28 selected, stride : 4347, ticket : 23, pass : 927 -> 5274 (16/20)
Process 29 selected, stride : 3571, ticket : 28, pass : 3571 -> 7142 (20/20)
Process 29 exit
Process 28 selected, stride : 4347, ticket : 23, pass : 5274 -> 9621 (17/20)
Process 27 selected, stride : 9090, ticket : 11, pass : 7500 -> 16590 (9/20)
Rebase Process Start
Process 27's pass is standardize from 16590, to 6969
Process 28's pass is standardize from 9621, to 0
Rebase Process End
Process 28 selected, stride : 4347, ticket : 23, pass : 0 -> 4347 (18/20)
Process 28 selected, stride : 4347, ticket : 23, pass : 4347 -> 8694 (19/20)
Process 27 selected, stride : 9090, ticket : 11, pass : 6969 -> 16059 (10/20)
Rebase Process Start
Process 27's pass is standardize from 16059, to 7365
Process 28's pass is standardize from 8694, to 0
Rebase Process End
Process 28 selected, stride : 4347, ticket : 23, pass : 0 -> 4347 (20/20)
Process 28 exit
Process 27 selected, stride : 9090, ticket : 11, pass : 7365 -> 16455 (11/20)
Rebase Process Start
Process 27's pass is standardize from 16455, to 0
Rebase Process End
Process 27 selected, stride : 9090, ticket : 11, pass : 0 -> 9090 (12/20)
Process 27 selected, stride : 9090, ticket : 11, pass : 9090 -> 18180 (13/20)
Rebase Process Start
Process 27's pass is standardize from 18180, to 0
Rebase Process End
Process 27 selected, stride : 9090, ticket : 11, pass : 0 -> 9090 (14/20)
Process 27 selected, stride : 9090, ticket : 11, pass : 9090 -> 18180 (15/20)
Rebase Process Start
Process 27's pass is standardize from 18180, to 0
Rebase Process End
Process 27 selected, stride : 9090, ticket : 11, pass : 0 -> 9090 (16/20)
Process 27 selected, stride : 9090, ticket : 11, pass : 9090 -> 18180 (17/20)
Rebase Process Start
Process 27's pass is standardize from 18180, to 0
Rebase Process End
Process 27 selected, stride : 9090, ticket : 11, pass : 0 -> 9090 (18/20)
Process 27 selected, stride : 9090, ticket : 11, pass : 9090 -> 18180 (19/20)
Rebase Process Start
```

```
Rebase Process Start
Process 27's pass is standardize from 18180, to 0
Rebase Process End
Process 27 selected, stride : 9090, ticket : 11, pass : 0 -> 9090 (20/20)
Process 27 exit

test4 end
```

test4는 서로 다른 티켓을 가진 3개 프로세스(pid 27, 28, 29)를 Stride로 스케줄링 할 때,pass값에 따른 rebase를 섞어가며 어떻게 스케줄링 되는지를 보여준다. pid 27, 28, 29 순으로 stride가 P27 > P28 > P29임을 관찰 할 수 있다. 따라서 종료되는 순서는 P29 -> P28 -> P27 순으로 종료될 것임을 예상할 수 있다. 이때 출력을 보면 “Process 27’s pass is standardize from 18180, to 0” 등의 출력을 관찰할 수 있는데, 이는 Rebase 했음을 의미한다. Rebase란 pass 값이 너무 커지거나(PASS_MAX=15000 초과), 프로세스 간 격차가 너무 커질 때(DISTANCE_MAX) 실행하는 정규화 단계이다.
구체적인 동작은, 모든 RUNNABLE 프로세스의 pass에서 최서 pass(min_pass)를 빼서 최소값을 0으로 맞춘다. 즉 상대적 순서는 그대로 유지하면서, 숫자만 줄여 오버플로/큰 수 문제를 방지한다. 일부 문구에는 “without distance cutting, to 7500 같은 문구가 보이는데, 이는 프로세스 간의 pass 값 차이가 DISTANCE_MAX = 7500 이상 벌어지면, 격차를 잘라서 유지하도록 구현된 로직이다.

[test 5 결과]

```
test5 start

Process 30 start
Process 31 start
Process 32 start
Process 30 selected, stride : 11111, ticket : 9, pass : 0 -> 11111 (1/10)
Process 31 selected, stride : 3703, ticket : 27, pass : 0 -> 3703 (1/10)
Process 32 selected, stride : 2325, ticket : 43, pass : 0 -> 2325 (1/10)
Process 32 selected, stride : 2325, ticket : 43, pass : 2325 -> 4650 (2/10)
Process 31 selected, stride : 3703, ticket : 27, pass : 3703 -> 7406 (2/10)
Process 32 selected, stride : 2325, ticket : 43, pass : 4650 -> 6975 (3/10)
Process 32 selected, stride : 2325, ticket : 43, pass : 6975 -> 9300 (4/10)
Process 31 selected, stride : 3703, ticket : 27, pass : 7406 -> 11109 (3/10)
Process 32 selected, stride : 2325, ticket : 43, pass : 9300 -> 11625 (5/10)
Process 31 selected, stride : 3703, ticket : 27, pass : 11109 -> 14812 (4/10)
Process 30 selected, stride : 11111, ticket : 9, pass : 11111 -> 22222 (2/10)
Rebase Process Start
Process 30's pass is standardize from 22222, with distance cutting, to 7500
Process 31's pass is standardize from 14812, to 3187
Process 32's pass is standardize from 11625, to 0
Rebase Process End
Process 32 selected, stride : 2325, ticket : 43, pass : 0 -> 2325 (6/10)
Process 32 selected, stride : 2325, ticket : 43, pass : 2325 -> 4650 (7/10)
Process 31 selected, stride : 3703, ticket : 27, pass : 3187 -> 6890 (5/10)
Process 32 selected, stride : 2325, ticket : 43, pass : 4650 -> 6975 (8/10)
Process 31 selected, stride : 3703, ticket : 27, pass : 6890 -> 10593 (6/10)
Process 32 selected, stride : 2325, ticket : 43, pass : 6975 -> 9300 (9/10)
Process 30 selected, stride : 11111, ticket : 9, pass : 7500 -> 18611 (3/10)
Rebase Process Start
Process 30's pass is standardize from 18611, with distance cutting, to 7500
Process 31's pass is standardize from 10593, to 1293
Process 32's pass is standardize from 9300, to 0
Rebase Process End
Process 32 selected, stride : 2325, ticket : 43, pass : 0 -> 2325 (10/10)
Process 32 exit
Process 33 start
Process 34 start
Process 33 selected, stride : 7142, ticket : 14, pass : 0 -> 7142 (1/10)
Process 34 selected, stride : 1639, ticket : 61, pass : 0 -> 1639 (1/10)
```

```
Process 34 selected, stride : 1639, ticket : 61, pass : 0 -> 1639 (1/10)
Process 31 selected, stride : 3703, ticket : 27, pass : 1293 -> 4996 (7/10)
Process 34 selected, stride : 1639, ticket : 61, pass : 1639 -> 3278 (2/10)
Process 34 selected, stride : 1639, ticket : 61, pass : 3278 -> 4917 (3/10)
Process 34 selected, stride : 1639, ticket : 61, pass : 4917 -> 6556 (4/10)
Process 31 selected, stride : 3703, ticket : 27, pass : 4996 -> 8699 (8/10)
Process 34 selected, stride : 1639, ticket : 61, pass : 6556 -> 8195 (5/10)
Process 33 selected, stride : 7142, ticket : 14, pass : 7142 -> 14284 (2/10)
Process 30 selected, stride : 11111, ticket : 9, pass : 7500 -> 18611 (4/10)
Rebase Process Start
Process 30's pass is standardize from 18611, with distance cutting, to 7500
Process 31's pass is standardize from 8699, to 504
Process 33's pass is standardize from 14284, to 6089
Process 34's pass is standardize from 8195, to 0
Rebase Process End
Process 34 selected, stride : 1639, ticket : 61, pass : 0 -> 1639 (6/10)
Process 31 selected, stride : 3703, ticket : 27, pass : 504 -> 4207 (9/10)
Process 34 selected, stride : 1639, ticket : 61, pass : 1639 -> 3278 (7/10)
Process 34 selected, stride : 1639, ticket : 61, pass : 3278 -> 4917 (8/10)
Process 31 selected, stride : 3703, ticket : 27, pass : 4207 -> 7910 (10/10)
Process 31 exit
Process 34 selected, stride : 1639, ticket : 61, pass : 4917 -> 6556 (9/10)
Process 33 selected, stride : 7142, ticket : 14, pass : 6089 -> 13231 (3/10)
Process 34 selected, stride : 1639, ticket : 61, pass : 6556 -> 8195 (10/10)
Process 34 exit
Process 35 start
Process 35 selected, stride : 1298, ticket : 77, pass : 0 -> 1298 (1/10)
Process 35 selected, stride : 1298, ticket : 77, pass : 1298 -> 2596 (2/10)
Process 35 selected, stride : 1298, ticket : 77, pass : 2596 -> 3894 (3/10)
Process 35 selected, stride : 1298, ticket : 77, pass : 3894 -> 5192 (4/10)
Process 35 selected, stride : 1298, ticket : 77, pass : 5192 -> 6490 (5/10)
Process 35 selected, stride : 1298, ticket : 77, pass : 6490 -> 7788 (6/10)
Process 30 selected, stride : 11111, ticket : 9, pass : 7500 -> 18611 (5/10)
Rebase Process Start
Process 30's pass is standardize from 18611, with distance cutting, to 7500
Process 35's pass is standardize from 7788, to 0
Process 33's pass is standardize from 13231, to 5443
Rebase Process End
Process 35 selected, stride : 1298, ticket : 77, pass : 0 -> 1298 (7/10)
Process 35 selected, stride : 1298, ticket : 77, pass : 1298 -> 2596 (8/10)
```

```
Process 35 selected, stride : 1298, ticket
Process 35 selected, stride : 1298, ticket
Process 35 selected, stride : 1298, ticket
Process 35 exit
Process 33 selected, stride : 7142, ticket
Process 30 selected, stride : 11111, ticke
Rebase Process Start
Process 30's pass is standardize from 1861
Process 33's pass is standardize from 1258
Rebase Process End
Process 33 selected, stride : 7142, ticket
Process 30 selected, stride : 11111, ticke
Rebase Process Start
Process 30's pass is standardize from 1713
Process 33's pass is standardize from 7142
Rebase Process End
Process 33 selected, stride : 7142, ticket
Process 33 selected, stride : 7142, ticket
Process 30 selected, stride : 11111, ticke
Rebase Process Start
Process 30's pass is standardize from 1861
Process 33's pass is standardize from 1428
Rebase Process End
Process 33 selected, stride : 7142, ticket
Process 30 selected, stride : 11111, ticke
Rebase Process Start
Process 30's pass is standardize from 1543
Process 33's pass is standardize from 7142
```

```
Rebase Process End
Process 33 selected, stride : 7142, ticket : 14, pass : 0 -> 7142 (5/10)
Process 30 selected, stride : 11111, ticket : 9, pass : 6026 -> 17137 (7/10)
Rebase Process Start
Process 30's pass is standardize from 17137, with distance cutting, to 7500
Process 33's pass is standardize from 7142, to 0
Rebase Process End
Process 33 selected, stride : 7142, ticket : 14, pass : 0 -> 7142 (6/10)
Process 33 selected, stride : 7142, ticket : 14, pass : 7142 -> 14284 (7/10)
Process 30 selected, stride : 11111, ticket : 9, pass : 7500 -> 18611 (8/10)
Rebase Process Start
Process 30's pass is standardize from 18611, to 4327
Process 33's pass is standardize from 14284, to 0
Rebase Process End
Process 33 selected, stride : 7142, ticket : 14, pass : 0 -> 7142 (8/10)
Process 30 selected, stride : 11111, ticket : 9, pass : 4327 -> 15438 (9/10)
Rebase Process Start
Process 30's pass is standardize from 15438, with distance cutting, to 7500
Process 33's pass is standardize from 7142, to 0
Rebase Process End
Process 33 selected, stride : 7142, ticket : 14, pass : 0 -> 7142 (9/10)
Process 33 selected, stride : 7142, ticket : 14, pass : 7142 -> 14284 (10/10)
Process 33 exit
Process 30 selected, stride : 11111, ticket : 9, pass : 7500 -> 18611 (10/10)
Process 30 exit
test5 end
```

test5는 티켓이 서로 다른 6개 프로세스를 돌려서, 티켓이 큰 프로세스일수록 더 자주 선택되어 먼저 10틱을 채우고 종료하고, 중간에 rebase로 pass 숫자를 줄여 overflow/과도한 격차를 방지한다는 걸 보여준다. test5에서도 알 수 있듯이, 선택 빈도는 티켓값에 비례하고, rebase의 목표는 숫자 안정성(오버플로/너무 큰 수 방지)이라는 것을 알 수 있다.

4. 소스코드

9개의 수정한 소스코드 + 테스트 프로그램 3개를 함께 소스코드 폴더에 첨부하였습니다.

테스트 프로그램이 없으면 Makefile UPROGS 규칙과 맞지 않아 make qemu-nox가 안되기 때문입니다.

```
objdump -S _debug_test > debug_test.asm
objdump -t _debug_test | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > debug_test.sym
make: *** 'fs.img'에서 필요한 '_syscall_test' 타겟을 만들 규칙이 없습니다.  멈춤.
```

(예시) _syscall_test가 없이 make qemu-nox를 하는 경우

[Makefile]

```
OBJS = \
	bio.o\
	console.o\
	exec.o\
	file.o\
	fs.o\
	ide.o\
	ioapic.o\
	kalloc.o\
	kbd.o\
	lapic.o\
	log.o\
	main.o\
	mp.o\
	picirq.o\
	pipe.o\
	proc.o\
	sleeplock.o\
	spinlock.o\
	string.o\
	swtch.o\
	syscall.o\
	sysfile.o\
	sysproc.o\
	trapasm.o\
	trap.o\
	uart.o\
	vectors.o\
	vm.o\

# Cross-compiling (e.g., on Mac OS X)
# TOOLPREFIX = i386-jos-elf

# Using native tools (e.g., on X86 Linux)
#TOOLPREFIX =

# Try to infer the correct TOOLPREFIX if not set
ifndef TOOLPREFIX
TOOLPREFIX := $(shell if i386-jos-elf-objdump -i 2>&1 | grep '^elf32-i386$$' >/dev/null 2>&1; \
	then echo 'i386-jos-elf-'; \
	elif objdump -i 2>&1 | grep 'elf32-i386' >/dev/null 2>&1; \
	then echo ''; \
	else echo "***" 1>&2; \
	echo "*** Error: Couldn't find an i386-*-elf version of GCC/binutils." 1>&2; \
	echo "*** Is the directory with i386-jos-elf-gcc in your PATH?" 1>&2; \
	echo "*** If your i386-*-elf toolchain is installed with a command" 1>&2; \
	echo "*** prefix other than 'i386-jos-elf-', set your TOOLPREFIX" 1>&2; \
	echo "*** environment variable to that prefix and run 'make' again." 1>&2; \
```

```
	echo "*** To turn off this error, run 'gmake TOOLPREFIX= ...'." 1>&2; \
	echo "***" 1>&2; exit 1; fi)
endif

# If the makefile can't find QEMU, specify its path here
# QEMU = qemu-system-i386

# Try to infer the correct QEMU
ifndef QEMU
QEMU = $(shell if which qemu > /dev/null; \
	then echo qemu; exit; \
	elif which qemu-system-i386 > /dev/null; \
	then echo qemu-system-i386; exit; \
	elif which qemu-system-x86_64 > /dev/null; \
	then echo qemu-system-x86_64; exit; \
	else \
	qemu=/Applications/Q.app/Contents/MacOS/i386-softmmu.app/Contents/MacOS/i386-softmmu; \
	if test -x $$qemu; then echo $$qemu; exit; fi; fi; \
	echo "***" 1>&2; \
	echo "*** Error: Couldn't find a working QEMU executable." 1>&2; \
	echo "*** Is the directory containing the qemu binary in your PATH" 1>&2; \
	echo "*** or have you tried setting the QEMU variable in Makefile?" 1>&2; \
	echo "***" 1>&2; exit 1)
endif

CC = $(TOOLPREFIX)gcc
AS = $(TOOLPREFIX)gas
LD = $(TOOLPREFIX)ld
OBJCOPY = $(TOOLPREFIX)objcopy
OBJDUMP = $(TOOLPREFIX)objdump
CFLAGS = -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb -m32 -Werror
-fno-omit-frame-pointer
CFLAGS += $(shell $(CC) -fno-stack-protector -E -x c /dev/null >/dev/null 2>&1 && echo -fno-stack-protector)
ASFLAGS = -m32 -gdwarf-2 -Wa,-divide
# FreeBSD ld wants ``elf_i386_fbsd''
LDFLAGS += -m $(shell $(LD) -V | grep elf_i386 2>/dev/null | head -n 1)

# Disable PIE when possible (for Ubuntu 16.10 toolchain)
ifneq ($(shell $(CC) -dumpspecs 2>/dev/null | grep -e '[^f]no-pie'),)
CFLAGS += -fno-pie -no-pie
endif
ifneq ($(shell $(CC) -dumpspecs 2>/dev/null | grep -e '[^f]nopie'),)
CFLAGS += -fno-pie -nopie
endif

xv6.img: bootblock kernel
	dd if=/dev/zero of=xv6.img count=10000
	dd if=bootblock of=xv6.img conv=notrunc
	dd if=kernel of=xv6.img seek=1 conv=notrunc

xv6memfs.img: bootblock kernelmemfs
	dd if=/dev/zero of=xv6memfs.img count=10000
	dd if=bootblock of=xv6memfs.img conv=notrunc
	dd if=kernelmemfs of=xv6memfs.img seek=1 conv=notrunc

bootblock: bootasm.S bootmain.c
	$(CC) $(CFLAGS) -fno-pic -O -nostdinc -I. -c bootmain.c
	$(CC) $(CFLAGS) -fno-pic -nostdinc -I. -c bootasm.S
```

```
	$(LD) $(LDFLAGS) -N -e start -Ttext 0x7C00 -o bootblock.o bootasm.o bootmain.o
	$(OBJDUMP) -S bootblock.o > bootblock.asm
	$(OBJCOPY) -S -O binary -j .text bootblock.o bootblock
	./sign.pl bootblock

entryother: entryother.S
	$(CC) $(CFLAGS) -fno-pic -nostdinc -I. -c entryother.S
	$(LD) $(LDFLAGS) -N -e start -Ttext 0x7000 -o bootblockother.o entryother.o
	$(OBJCOPY) -S -O binary -j .text bootblockother.o entryother
	$(OBJDUMP) -S bootblockother.o > entryother.asm

initcode: initcode.S
	$(CC) $(CFLAGS) -nostdinc -I. -c initcode.S
	$(LD) $(LDFLAGS) -N -e start -Ttext 0 -o initcode.out initcode.o
	$(OBJCOPY) -S -O binary initcode.out initcode
	$(OBJDUMP) -S initcode.o > initcode.asm

kernel: $(OBJS) entry.o entryother initcode kernel.ld
	$(LD) $(LDFLAGS) -T kernel.ld -o kernel entry.o $(OBJS) -b binary initcode entryother
	$(OBJDUMP) -S kernel > kernel.asm
	$(OBJDUMP) -t kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > kernel.sym

# kernelmemfs is a copy of kernel that maintains the
# disk image in memory instead of writing to a disk.
# This is not so useful for testing persistent storage or
# exploring disk buffering implementations, but it is
# great for testing the kernel on real hardware without
# needing a scratch disk.
MEMFSOBJS = $(filter-out ide.o,$(OBJS)) memide.o
kernelmemfs: $(MEMFSOBJS) entry.o entryother initcode kernel.ld fs.img
	$(LD) $(LDFLAGS) -T kernel.ld -o kernelmemfs entry.o  $(MEMFSOBJS) -b binary initcode entryother fs.img
	$(OBJDUMP) -S kernelmemfs > kernelmemfs.asm
	$(OBJDUMP) -t kernelmemfs | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > kernelmemfs.sym

tags: $(OBJS) entryother.S _init
	etags *.S *.c

vectors.S: vectors.pl
	./vectors.pl > vectors.S

ULIB = ulib.o usys.o printf.o umalloc.o

_%: %.o $(ULIB)
	$(LD) $(LDFLAGS) -N -e main -Ttext 0 -o $@ $^
	$(OBJDUMP) -S $@ > $*.asm
	$(OBJDUMP) -t $@ | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > $*.sym

_forktest: forktest.o $(ULIB)
	# forktest has less library code linked in - needs to be small
	# in order to be able to max out the proc table.
	$(LD) $(LDFLAGS) -N -e main -Ttext 0 -o _forktest forktest.o ulib.o usys.o
	$(OBJDUMP) -S _forktest > forktest.asm

mkfs: mkfs.c fs.h
	gcc -Werror -Wall -o mkfs mkfs.c

# Prevent deletion of intermediate files, e.g. cat.o, after first build, so
# that disk image changes after first build are persistent until clean.  More
```

```
# details:
# http://www.gnu.org/software/make/manual/html_node/Chained-Rules.html
.PRECIOUS: %.o

UPROGS=\
	_cat\
	_echo\
	_forktest\
	_grep\
	_init\
	_kill\
	_ln\
	_ls\
	_mkdir\
	_rm\
	_sh\
	_stressfs\
	_usertests\
	_wc\
	_zombie\
	_debug_test\
	_syscall_test\
	_scheduler_test


fs.img: mkfs README $(UPROGS)
	./mkfs fs.img README $(UPROGS)

-include *.d

clean:
	rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
	*.o *.d *.asm *.sym vectors.S bootblock entryother \
	initcode initcode.out kernel xv6.img fs.img kernelmemfs \
	xv6memfs.img mkfs .gdbinit \
	$(UPROGS)

# make a printout
FILES = $(shell grep -v '^\#' runoff.list)
PRINT = runoff.list runoff.spec README toc.hdr toc.ftr $(FILES)

xv6.pdf: $(PRINT)
	./runoff
	ls -l xv6.pdf

print: xv6.pdf

# run in emulators

bochs : fs.img xv6.img
	if [ ! -e .bochsrc ]; then ln -s dot-bochsrc .bochsrc; fi
	bochs -q

# try to generate a unique GDB port
GDBPORT = $(shell expr `id -u` % 5000 + 25000)
# QEMU's gdb stub command line changed in 0.11
QEMUGDB = $(shell if $(QEMU) -help | grep -q '^-gdb'; \
	then echo "-gdb tcp::$(GDBPORT)"; \
```

```
        else echo "-s -p $(GDBPORT)"; fi)
ifndef CPUS
CPUS := 2
endif
QEMUOPTS = -drive file=fs.img,index=1,media=disk,format=raw -drive file=xv6.img,index=0,media=disk,format=raw -smp
$(CPUS) -m 512 $(QEMUEXTRA)

qemu: fs.img xv6.img
        $(QEMU) -serial mon:stdio $(QEMUOPTS)

qemu-memfs: xv6memfs.img
        $(QEMU) -drive file=xv6memfs.img,index=0,media=disk,format=raw -smp $(CPUS) -m 256

qemu-nox: fs.img xv6.img
        $(QEMU) -nographic $(QEMUOPTS)

.gdbinit: .gdbinit.tmpl
        sed "s/localhost:1234/localhost:$(GDBPORT)/" < $^ > $@

qemu-gdb: fs.img xv6.img .gdbinit
        @echo "*** Now run 'gdb'." 1>&2
        $(QEMU) -serial mon:stdio $(QEMUOPTS) -S $(QEMUGDB)

qemu-nox-gdb: fs.img xv6.img .gdbinit
        @echo "*** Now run 'gdb'." 1>&2
        $(QEMU) -nographic $(QEMUOPTS) -S $(QEMUGDB)

# CUT HERE
# prepare dist for students
# after running make dist, probably want to
# rename it to rev0 or rev1 or so on and then
# check in that version.

EXTRA=\
        mkfs.c ulib.c user.h cat.c echo.c forktest.c grep.c kill.c\
        ln.c ls.c mkdir.c rm.c stressfs.c usertests.c wc.c zombie.c\
        printf.c umalloc.c\
        README dot-bochsrc *.pl toc.* runoff runoff1 runoff.list\
        .gdbinit.tmpl gdbutil\

dist:
        rm -rf dist
        mkdir dist
        for i in $(FILES); \
        do \
                grep -v PAGEBREAK $$i >dist/$$i; \
        done
        sed '/CUT HERE/,$$d' Makefile >dist/Makefile
        echo >dist/runoff.spec
        cp $(EXTRA) dist

dist-test:
        rm -rf dist
        make dist
        rm -rf dist-test
        mkdir dist-test
        cp dist/* dist-test
        cd dist-test; $(MAKE) print
```

```
	cd dist-test; $(MAKE) bochs || true
	cd dist-test; $(MAKE) qemu

# update this rule (change rev#) when it is time to
# make a new revision.
tar:
	rm -rf /tmp/xv6
	mkdir -p /tmp/xv6
	cp dist/* dist/.gdbinit.tmpl /tmp/xv6
	(cd /tmp; tar cf - xv6) | gzip >xv6-rev10.tar.gz  # the next one will be 10 (9/17)

.PHONY: dist-test dist
```

[proc.c]

```
#include "types.h"
#include "defs.h"
#include "param.h"
#include "memlayout.h"
#include "mmu.h"
#include "x86.h"
#include "proc.h"
#include "spinlock.h"

struct {
  struct spinlock lock;
  struct proc proc[NPROC];
} ptable;

static struct proc *initproc;

int nextpid = 1;
extern void forkret(void);
extern void trapret(void);

static void wakeup1(void *chan);

// Stride 스케줄러 헬퍼 함수 원형
static struct proc* pick_min_pass_proc(void);
static void rebase_passes(void);

void
pinit(void)
{
  initlock(&ptable.lock, "ptable");
}

// Must be called with interrupts disabled
int
cpuid() {
  return mycpu()-cpus;
}

// Must be called with interrupts disabled to avoid the caller being
// rescheduled between reading lapicid and running through the loop.
struct cpu*
mycpu(void)
{
  int apicid, i;
```

```
  if(readeflags()&FL_IF)
    panic("mycpu called with interrupts enabled\n");

  apicid = lapicid();
  // APIC IDs are not guaranteed to be contiguous. Maybe we should have
  // a reverse map, or reserve a register to store &cpus[i].
  for (i = 0; i < ncpu; ++i) {
    if (cpus[i].apicid == apicid)
      return &cpus[i];
  }
  panic("unknown apicid\n");
}

// Disable interrupts so that we are not rescheduled
// while reading proc from the cpu structure
struct proc*
myproc(void) {
  struct cpu *c;
  struct proc *p;
  pushcli();
  c = mycpu();
  p = c->proc;
  popcli();
  return p;
}

//PAGEBREAK: 32
// Look in the process table for an UNUSED proc.
// If found, change state to EMBRYO and initialize
// state required to run in the kernel.
// Otherwise return 0.
static struct proc*
allocproc(void)
{
  struct proc *p;
  char *sp;

  // 전역 프로세스 테이블(배열)을 훑어 UNUSED 상태를 찾을 때, 동시성 문제 있으므로 ptable.lock을 잡고 진행한다
  acquire(&ptable.lock);

  for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
    if(p->state == UNUSED)
      goto found;

  // UNUSED 슬롯을 못 찾으면 종료(return 0)
  release(&ptable.lock);
  return 0;


found:
  p->state = EMBRYO; // UNUSED를 찾는 순간 EMBRYO로 상태를 바꾼다 -> 슬롯 사용 표시만 한 것. 아직 유저 프로그램을 돌릴 수 있는 상태는 아님
  p->pid = nextpid++;  // pid 할당

  release(&ptable.lock); // 테이블 갱신이 끝났으므로 락 해제

  // Allocate kernel stack.
```

```
  if((p->kstack = kalloc()) == 0){
        // 실패하면 슬롯 상태를 UNUSED로 롤백하고 종료
    p->state = UNUSED;
    return 0;
  }
  // 스택은 상위 주소에서 아래로 자란다. 따라서 스택 포인터 sp를 스택 최상단(높은 주소)로 잡는다.
  sp = p->kstack + KSTACKSIZE;

  // Leave room for trap frame.
  // 트랩 프레임(trapframe) 자리 잡기
  // trapframe: 시스템 콜/인터럽트 등으로 인한 유저 -> 커널 진입 시 유저 레지스터 상태를 저장하는 구조체이다.
  sp -= sizeof *p->tf;
  p->tf = (struct trapframe*)sp;

  // Set up new context to start executing at forkret,
  // which returns to trapret.
  // trapret: 트랩프레임을 복원해 유저 모드로 되돌리는 표준 루틴이다
  sp -= 4;
  *(uint*)sp = (uint)trapret;

  // 스케줄러가 swtch()로 이 프로세스에 처음으로 컨텍스트를 넘겨줄 때, 어디서부터 실행을 시작할지를 정한다
  // 그 시작점이 바로 forkret이다
  // forkret 안에서는 보통 락 해제 등 마무리를 하고, 이어서 위에 푸시해둔 trapret으로 점프해 유저/커널 복귀 흐름을
완성한다
  sp -= sizeof *p->context;
  p->context = (struct context*)sp;
  memset(p->context, 0, sizeof *p->context);
  p->context->eip = (uint)forkret;

  // stride scheduler 관련 초기화 추가
  p->tickets  = 1;  // 기본 티켓 수 1
  p->stride   = 0;  // settickets() 호출 시 계산
  p->pass     = 0;  // 시작 시 pass = 0
  p->ticks    = 1;  // 실행된 틱 누적
  p->end_ticks = -1; // 무제한 실행(end_ticks 설정 전까지)

  return p;
}

//PAGEBREAK: 32
// Set up first user process.
void
userinit(void)
{
  struct proc *p;
  extern char _binary_initcode_start[], _binary_initcode_size[];

  p = allocproc();

  initproc = p;
  if((p->pgdir = setupkvm()) == 0)
    panic("userinit: out of memory?");
  inituvm(p->pgdir, _binary_initcode_start, (int)_binary_initcode_size);
  p->sz = PGSIZE;
  memset(p->tf, 0, sizeof(*p->tf));
  p->tf->cs = (SEG_UCODE << 3) | DPL_USER;
  p->tf->ds = (SEG_UDATA << 3) | DPL_USER;
  p->tf->es = p->tf->ds;
```

```
  p->tf->ss = p->tf->ds;
  p->tf->eflags = FL_IF;
  p->tf->esp = PGSIZE;
  p->tf->eip = 0;  // beginning of initcode.S

  safestrcpy(p->name, "initcode", sizeof(p->name));
  p->cwd = namei("/");

  // this assignment to p->state lets other cores
  // run this process. the acquire forces the above
  // writes to be visible, and the lock is also needed
  // because the assignment might not be atomic.
  acquire(&ptable.lock);

  p->state = RUNNABLE;

  release(&ptable.lock);
}

// Grow current process's memory by n bytes.
// Return 0 on success, -1 on failure.
int
growproc(int n)
{
  uint sz;
  struct proc *curproc = myproc();

  sz = curproc->sz;
  if(n > 0){
    if((sz = allocuvm(curproc->pgdir, sz, sz + n)) == 0)
      return -1;
  } else if(n < 0){
    if((sz = deallocuvm(curproc->pgdir, sz, sz + n)) == 0)
      return -1;
  }
  curproc->sz = sz;
  switchuvm(curproc);
  return 0;
}

// Create a new process copying p as the parent.
// Sets up stack to return as if from system call.
// Caller must set state of returned proc to RUNNABLE.
int
fork(void)
{
  int i, pid;
  struct proc *np;
  struct proc *curproc = myproc();

  // Allocate process.
  if((np = allocproc()) == 0){
    return -1;
  }

  // Copy process state from proc.
  if((np->pgdir = copyuvm(curproc->pgdir, curproc->sz)) == 0){
    kfree(np->kstack);
```

```
    np->kstack = 0;
    np->state = UNUSED;
    return -1;
  }
  np->sz = curproc->sz;
  np->parent = curproc;
  *np->tf = *curproc->tf;

  // Clear %eax so that fork returns 0 in the child.
  np->tf->eax = 0;

  for(i = 0; i < NOFILE; i++)
    if(curproc->ofile[i])
      np->ofile[i] = filedup(curproc->ofile[i]);
  np->cwd = idup(curproc->cwd);

  safestrcpy(np->name, curproc->name, sizeof(curproc->name));

  pid = np->pid;

  acquire(&ptable.lock);

  np->state = RUNNABLE;

  if(np->pid > 2 && np->parent->pid > 2){
                    cprintf("Process %d start\n", np->pid);
  }

  release(&ptable.lock);

  return pid;
}

// Exit the current process.  Does not return.
// An exited process remains in the zombie state
// until its parent calls wait() to find out it exited.
void
exit(void)
{
  struct proc *curproc = myproc();
  struct proc *p;
  int fd;

  if(curproc == initproc)
    panic("init exiting");

  // Close all open files.
  for(fd = 0; fd < NOFILE; fd++){
    if(curproc->ofile[fd]){
      fileclose(curproc->ofile[fd]);
      curproc->ofile[fd] = 0;
    }
  }

  begin_op();
  iput(curproc->cwd);
  end_op();
  curproc->cwd = 0;
```

```
  acquire(&ptable.lock);

  // 종료되는 프로세스의 pass를 0으로 초기화
  curproc->pass = 0;

  // Parent might be sleeping in wait().
  wakeup1(curproc->parent);

  // Pass abandoned children to init.
  for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
    if(p->parent == curproc){
      p->parent = initproc;
      if(p->state == ZOMBIE)
        wakeup1(initproc);
    }
  }

  if(curproc->pid > 2 && curproc->parent->pid > 2){
                      cprintf("Process %d exit\n", curproc->pid);
  }

  // Jump into the scheduler, never to return.
  curproc->state = ZOMBIE;
  sched();
  panic("zombie exit");
}

// Wait for a child process to exit and return its pid.
// Return -1 if this process has no children.
int
wait(void)
{
  struct proc *p;
  int havekids, pid;
  struct proc *curproc = myproc();

  acquire(&ptable.lock);
  for(;;){
    // Scan through table looking for exited children.
    havekids = 0;
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
      if(p->parent != curproc)
        continue;
      havekids = 1;
      if(p->state == ZOMBIE){
        // Found one.
        pid = p->pid;
        kfree(p->kstack);
        p->kstack = 0;
        freevm(p->pgdir);
        p->pid = 0;
        p->parent = 0;
        p->name[0] = 0;
        p->killed = 0;
        p->state = UNUSED;
        release(&ptable.lock);
        return pid;
```

```
      }
    }

    // No point waiting if we don't have any children.
    if(!havekids || curproc->killed){
      release(&ptable.lock);
      return -1;
    }

    // Wait for children to exit.  (See wakeup1 call in proc_exit.)
    sleep(curproc, &ptable.lock);  //DOC: wait-sleep
  }
}

//PAGEBREAK: 42
// Per-CPU process scheduler.
// Each CPU calls scheduler() after setting itself up.
// Scheduler never returns.  It loops, doing:
//  - choose a process to run
//  - swtch to start running that process
//  - eventually that process transfers control
//      via swtch back to the scheduler.
void
scheduler(void)
{
  struct cpu *c = mycpu();
  c->proc = 0;

  for(;;){
    // Enable interrupts on this processor.
    sti();

    // Loop over process table looking for process to run.
    acquire(&ptable.lock);

        // 1) RUNNABLE 중 최소 pass를 선택(pass 값이 같다면, 최소 pid를 선택)
        struct proc *p = pick_min_pass_proc();

        if(p == 0){
                        // 실행할 것이 없는 경우
                        release(&ptable.lock);
                        continue;
        }

        // 2) 선택된 프로세스에게 CPU 1틱 부여
        c->proc = p;
        switchuvm(p);             // 프로세스 p의 주소로 가상 주소를 보게 한다
        p->state = RUNNING;                 // 프로세스 p가 실제로 CPU를 쓸 수 있도록 RUNNING 상태로 바꾼다

        // 컨텍스트 스위치: p가 타이머에 선점되거나 yield/블록 되면 돌아온다
        swtch(&c->scheduler, p->context);  // 지금(스케줄러)의 register set을 c->scheduler 컨텍스트에 저장하고, 프로세스
쪽에 저장돼 있던 p->context를 복원해서 그 자리부터 이어서 실행한다
        // 즉 스케줄러쪽 코드는 멈추고, 프로세스 코드가 시작된다
        // 반대로, 나중에 p가 yield()/sleep() 하거나 타이머에 선점되면 sched() 내부에서 역방향으로 swtch(&p->context,
&c->scheduler)가 호출되어 스케줄러로 돌아온다
        switchkvm(); // 프로세스 실행을 마치고 스케줄러로 돌아온 직후, MMU를 커널 전용 페이지 테이블로 되돌린다.
        c->proc = 0;  // per-CPU 변수에 "현재 실행 중인 프로세스 없음"을 기록한다
```

```
      release(&ptable.lock);
    }
  }

// Enter scheduler.  Must hold only ptable.lock
// and have changed proc->state. Saves and restores
// intena because intena is a property of this
// kernel thread, not this CPU. It should
// be proc->intena and proc->ncli, but that would
// break in the few places where a lock is held but
// there's no process.
void
sched(void)
{
  int intena;
  struct proc *p = myproc();

  if(!holding(&ptable.lock))
    panic("sched ptable.lock");
  if(mycpu()->ncli != 1)
    panic("sched locks");
  if(p->state == RUNNING)
    panic("sched running");
  if(readeflags()&FL_IF)
    panic("sched interruptible");

    // 각 프로세스가 1틱만큼 CPU 점유를 마치고 제어권을 넘길 때 본인의 stride 값만큼 pass 값을 올림(명세)
     p->ticks++;
     p->pass += p->stride;

     // 프로세스의 pass값이 PASS_MAX 초과 시 rebase_passes() 호출
     if(p->pass > PASS_MAX){
             rebase_passes();
}

  intena = mycpu()->intena;
  swtch(&p->context, mycpu()->scheduler);
  mycpu()->intena = intena;
}

// Give up the CPU for one scheduling round.
void
yield(void)
{
  acquire(&ptable.lock);  //DOC: yieldlock
  myproc()->state = RUNNABLE;
  sched();
  release(&ptable.lock);
}

// A fork child's very first scheduling by scheduler()
// will swtch here.  "Return" to user space.
void
forkret(void)
{
  static int first = 1;
  // Still holding ptable.lock from scheduler.
```

```
  release(&ptable.lock);

  if (first) {
    // Some initialization functions must be run in the context
    // of a regular process (e.g., they call sleep), and thus cannot
    // be run from main().
    first = 0;
    iinit(ROOTDEV);
    initlog(ROOTDEV);
  }

  // Return to "caller", actually trapret (see allocproc).
}

// Atomically release lock and sleep on chan.
// Reacquires lock when awakened.
void
sleep(void *chan, struct spinlock *lk)
{
  struct proc *p = myproc();

  if(p == 0)
    panic("sleep");

  if(lk == 0)
    panic("sleep without lk");

  // Must acquire ptable.lock in order to
  // change p->state and then call sched.
  // Once we hold ptable.lock, we can be
  // guaranteed that we won't miss any wakeup
  // (wakeup runs with ptable.lock locked),
  // so it's okay to release lk.
  if(lk != &ptable.lock){  //DOC: sleeplock0
    acquire(&ptable.lock);  //DOC: sleeplock1
    release(lk);
  }
  // Go to sleep.
  p->chan = chan;
  p->state = SLEEPING;

  sched();

  // Tidy up.
  p->chan = 0;

  // Reacquire original lock.
  if(lk != &ptable.lock){  //DOC: sleeplock2
    release(&ptable.lock);
    acquire(lk);
  }
}

//PAGEBREAK!
// Wake up all processes sleeping on chan.
// The ptable lock must be held.
static void
wakeup1(void *chan)
```

```
{
  struct proc *p;

  for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
    if(p->state == SLEEPING && p->chan == chan)
      p->state = RUNNABLE;
}

// Wake up all processes sleeping on chan.
void
wakeup(void *chan)
{
  acquire(&ptable.lock);
  wakeup1(chan);
  release(&ptable.lock);
}

// Kill the process with the given pid.
// Process won't exit until it returns
// to user space (see trap in trap.c).
int
kill(int pid)
{
  struct proc *p;

  acquire(&ptable.lock);
  for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
    if(p->pid == pid){
      p->killed = 1;
      // Wake process from sleep if necessary.
      if(p->state == SLEEPING)
        p->state = RUNNABLE;
      release(&ptable.lock);
      return 0;
    }
  }
  release(&ptable.lock);
  return -1;
}

//PAGEBREAK: 36
// Print a process listing to console.  For debugging.
// Runs when user types ^P on console.
// No lock to avoid wedging a stuck machine further.
void
procdump(void)
{
  static char *states[] = {
  [UNUSED]    "unused",
  [EMBRYO]    "embryo",
  [SLEEPING]  "sleep ",
  [RUNNABLE]  "runble",
  [RUNNING]   "run   ",
  [ZOMBIE]    "zombie"
  };
  int i;
  struct proc *p;
  char *state;
```

```
  uint pc[10];

  for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
    if(p->state == UNUSED)
      continue;
    if(p->state >= 0 && p->state < NELEM(states) && states[p->state])
      state = states[p->state];
    else
      state = "???";
    cprintf("%d %s %s", p->pid, state, p->name);
    if(p->state == SLEEPING){
      getcallerpcs((uint*)p->context->ebp+2, pc);
      for(i=0; i<10 && pc[i] != 0; i++)
        cprintf(" %p", pc[i]);
    }
    cprintf("\n");
  }
}

// 선형 탐색으로 지금까지 본 pid의 pass들 중에서 가장 값이 작은 프로세스를 선택하는 함수
static struct proc* pick_min_pass_proc(void){
                // pass가 가장 작은 것이 제일 좋은 것이므로 best로 naming
                struct proc *p, *best = 0;
                for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
                                // RUNNABLE인 프로세스만 고려한다
                                if(p->state != RUNNABLE)
                                                continue;

                                // pass 최솟값 갖는 프로세스를 선정, 동률이면 pid 작은 것을 선택
                                if(best == 0 || p->pass < best->pass || (p->pass == best->pass && p->pid <
best->pid)){
                                                best = p;
                                }
                }
                return best;
}

// 프로세스들의 pass 값 안정성(숫자 안정성)을 위해 정규화하는 함수
// distance cutting, 정규화의 기능을 한다
static void rebase_passes(void){
                // 0xFFFFFFFF은 <limits.h>의 UINT_MAX와 같다
                // unsigned int의 최대값인데, 이는 아직 최소값을 못 찾은 상태를 표시하는 플래그 역할을 한다
                uint min_pass = 0xFFFFFFFF;
                struct proc *p;

                // 1) RUNNABLE 중 최소 pass 찾기
                for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
                                // RUNNABLE인 프로세스만 고려한다
                                if(p->state != RUNNABLE)
                                                continue;
                                if(p->pass < min_pass)
                                                min_pass = p->pass;
                }

                if(min_pass == 0xFFFFFFFF)
                                return;   //RUNNABLE 없음

                cprintf("\nRebase Process Start\n\n");
```

```
                // 2) 모든 RUNNABLE 표준화 + distance cutting
                for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
                        if(p->state != RUNNABLE)
                                continue;          // RUNNABLE 아니면 고려하지 않는다

                        uint old = p->pass;
                        uint newp = old - min_pass;    // 가장 작은 프로세스의 pass만큼 빼기

                        // 정규환한 프로세스의 pass값이 DISTANCE_MAX의 값을 넘으면, 해당 프로세스의
pass값을 DISTANCE_MAX값으로 바꿔버린다.(clamp)
                        // 이는 모든 프로세스 간의 pass값의 차이가 DISTANCE_MAX 값을 넘기지 않음을
의미한다.
                        // 왜냐하면 정규화 후에는 가장 작은 pass값이 0이고, 따라서 어떤 프로세스의 pass값이
DISTANCE_MAX 이상만 아니라면, 모든 프로세스 간의 pass값 차이가 DISTANCE_MAX 이하로 형성되기 때문이다.
                        if(newp > DISTANCE_MAX){
                                cprintf("Process %d's pass is standardize from %d, with distance
cutting, to %d\n", p->pid, (int)old, (int)DISTANCE_MAX);
                                p->pass = DISTANCE_MAX;
                        } else {
                        // Distance Cutting 상황이 아니라면, 원래대로 정규화만 진행한다.
                                if(old != newp) {
                                        cprintf("Process %d's pass is standardize from
%d, to %d\n", p->pid, (int)old, (int)newp);
                                }
                                p->pass = newp;
                        }
                }
                cprintf("\nRebase Process End\n\n");
}
```

[proc.h]

```
// Per-CPU state
struct cpu {
  uchar apicid;                // Local APIC ID
  struct context *scheduler;   // swtch() here to enter scheduler
  struct taskstate ts;         // Used by x86 to find stack for interrupt
  struct segdesc gdt[NSEGS];   // x86 global descriptor table
  volatile uint started;       // Has the CPU started?
  int ncli;                    // Depth of pushcli nesting.
  int intena;                  // Were interrupts enabled before pushcli?
  struct proc *proc;           // The process running on this cpu or null
};

extern struct cpu cpus[NCPU];
extern int ncpu;

//PAGEBREAK: 17
// Saved registers for kernel context switches.
// Don't need to save all the segment registers (%cs, etc),
// because they are constant across kernel contexts.
// Don't need to save %eax, %ecx, %edx, because the
// x86 convention is that the caller has saved them.
// Contexts are stored at the bottom of the stack they
// describe; the stack pointer is the address of the context.
// The layout of the context matches the layout of the stack in swtch.S
// at the "Switch stacks" comment. Switch doesn't save eip explicitly,
// but it is on the stack and allocproc() manipulates it.
```

```
struct context {
  uint edi;
  uint esi;
  uint ebx;
  uint ebp;
  uint eip;
};

enum procstate { UNUSED, EMBRYO, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

// Per-process state
struct proc {
  uint sz;                     // Size of process memory (bytes)
  pde_t* pgdir;                // Page table
  char *kstack;                // Bottom of kernel stack for this process
  enum procstate state;        // Process state
  int pid;                     // Process ID
  struct proc *parent;         // Parent process
  struct trapframe *tf;        // Trap frame for current syscall
  struct context *context;     // swtch() here to run process
  void *chan;                  // If non-zero, sleeping on chan
  int killed;                  // If non-zero, have been killed
  struct file *ofile[NOFILE];  // Open files

  struct inode *cwd;           // Current directory
  char name[16];               // Process name (debugging)

  int tickets;                                  // 기본값: 1
  uint stride;                                  // 기본값: 0(setticket으로 설정)
  uint pass;                                    // 기본값: 0(setticket으로 설정)
  int ticks;                                    // 기본값: 0
  int end_ticks;                        // 기본값: -1(양수인 경우 ticks 변수가 end_ticks 값이 되면 프로세스 종료)
};

#define STRIDE_MAX 100000
#define PASS_MAX 15000
#define DISTANCE_MAX 7500

// Process memory is laid out contiguously, low addresses first:
//   text
//   original data and bss
//   fixed-size stack
//   expandable heap
```

[syscall.c]

```
#include "types.h"
#include "defs.h"
#include "param.h"
#include "memlayout.h"
#include "mmu.h"
#include "proc.h"
#include "x86.h"
#include "syscall.h"

// User code makes a system call with INT T_SYSCALL.
// System call number in %eax.
// Arguments on the stack, from the user call to the C
// library system call function. The saved user %esp points
```

```
// to a saved program counter, and then the first argument.

// Fetch the int at addr from the current process.
int
fetchint(uint addr, int *ip)
{
  struct proc *curproc = myproc();

  if(addr >= curproc->sz || addr+4 > curproc->sz)
    return -1;
  *ip = *(int*)(addr);
  return 0;
}

// Fetch the nul-terminated string at addr from the current process.
// Doesn't actually copy the string - just sets *pp to point at it.
// Returns length of string, not including nul.
int
fetchstr(uint addr, char **pp)
{
  char *s, *ep;
  struct proc *curproc = myproc();

  if(addr >= curproc->sz)
    return -1;
  *pp = (char*)addr;
  ep = (char*)curproc->sz;
  for(s = *pp; s < ep; s++){
    if(*s == 0)
      return s - *pp;
  }
  return -1;
}

// Fetch the nth 32-bit system call argument.
int
argint(int n, int *ip)
{
  return fetchint((myproc()->tf->esp) + 4 + 4*n, ip);
}

// Fetch the nth word-sized system call argument as a pointer
// to a block of memory of size bytes.  Check that the pointer
// lies within the process address space.
int
argptr(int n, char **pp, int size)
{
  int i;
  struct proc *curproc = myproc();

  if(argint(n, &i) < 0)
    return -1;
  if(size < 0 || (uint)i >= curproc->sz || (uint)i+size > curproc->sz)
    return -1;
  *pp = (char*)i;
  return 0;
}
```

```
// Fetch the nth word-sized system call argument as a string pointer.
// Check that the pointer is valid and the string is nul-terminated.
// (There is no shared writable memory, so the string can't change
// between this check and being used by the kernel.)
int
argstr(int n, char **pp)
{
  int addr;
  if(argint(n, &addr) < 0)
    return -1;
  return fetchstr(addr, pp);
}

extern int sys_chdir(void);
extern int sys_close(void);
extern int sys_dup(void);
extern int sys_exec(void);
extern int sys_exit(void);
extern int sys_fork(void);
extern int sys_fstat(void);
extern int sys_getpid(void);
extern int sys_kill(void);
extern int sys_link(void);
extern int sys_mkdir(void);
extern int sys_mknod(void);
extern int sys_open(void);
extern int sys_pipe(void);
extern int sys_read(void);
extern int sys_sbrk(void);
extern int sys_sleep(void);
extern int sys_unlink(void);
extern int sys_wait(void);
extern int sys_write(void);
extern int sys_uptime(void);
extern int sys_settickets(void);

static int (*syscalls[])(void) = {
[SYS_fork]    sys_fork,
[SYS_exit]    sys_exit,
[SYS_wait]    sys_wait,
[SYS_pipe]    sys_pipe,
[SYS_read]    sys_read,
[SYS_kill]    sys_kill,
[SYS_exec]    sys_exec,
[SYS_fstat]   sys_fstat,
[SYS_chdir]   sys_chdir,
[SYS_dup]     sys_dup,
[SYS_getpid]  sys_getpid,
[SYS_sbrk]    sys_sbrk,
[SYS_sleep]   sys_sleep,
[SYS_uptime]  sys_uptime,
[SYS_open]    sys_open,
[SYS_write]   sys_write,
[SYS_mknod]   sys_mknod,
[SYS_unlink]  sys_unlink,
[SYS_link]    sys_link,
[SYS_mkdir]   sys_mkdir,
[SYS_close]   sys_close,
```

```
[SYS_settickets] sys_settickets,
};

void
syscall(void)
{
  int num;
  struct proc *curproc = myproc();

  num = curproc->tf->eax;
  if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
    curproc->tf->eax = syscalls[num]();
  } else {
    cprintf("%d %s: unknown sys call %d\n",
            curproc->pid, curproc->name, num);
    curproc->tf->eax = -1;
  }
}
```

[syscall.h]

```
// System call numbers
#define SYS_fork      1
#define SYS_exit      2
#define SYS_wait      3
#define SYS_pipe      4
#define SYS_read      5
#define SYS_kill      6
#define SYS_exec      7
#define SYS_fstat     8
#define SYS_chdir     9
#define SYS_dup       10
#define SYS_getpid    11
#define SYS_sbrk      12
#define SYS_sleep     13
#define SYS_uptime    14
#define SYS_open      15
#define SYS_write     16
#define SYS_mknod     17
#define SYS_unlink    18
#define SYS_link      19
#define SYS_mkdir     20
#define SYS_close     21
#define SYS_settickets 22
```

[sysproc.c]

```
#include "types.h"
#include "x86.h"
#include "defs.h"
#include "date.h"
#include "param.h"
#include "memlayout.h"
#include "mmu.h"
#include "proc.h"

int
sys_fork(void)
{
  return fork();
```

```
}

int
sys_exit(void)
{
  exit();
  return 0;  // not reached
}

int
sys_wait(void)
{
  return wait();
}

int
sys_kill(void)
{
  int pid;

  if(argint(0, &pid) < 0)
    return -1;
  return kill(pid);
}

int
sys_getpid(void)
{
  return myproc()->pid;
}

int
sys_sbrk(void)
{
  int addr;
  int n;

  if(argint(0, &n) < 0)
    return -1;
  addr = myproc()->sz;
  if(growproc(n) < 0)
    return -1;
  return addr;
}

int
sys_sleep(void)
{
  int n;
  uint ticks0;

  if(argint(0, &n) < 0)
    return -1;
  acquire(&tickslock);
  ticks0 = ticks;
  while(ticks - ticks0 < n){
    if(myproc()->killed){
      release(&tickslock);
```

```
      return -1;
    }
    sleep(&ticks, &tickslock);
  }
  release(&tickslock);
  return 0;
}

// return how many clock tick interrupts have occurred
// since start.
int
sys_uptime(void)
{
  uint xticks;

  acquire(&tickslock);
  xticks = ticks;
  release(&tickslock);
  return xticks;
}

// 커널 핸들러(시스템 콜)
// 티켓값이 클수록 stride가 작아지면서 더 자주 선택된다. 따라서 티켓값이 클수록(stride가 작을수록) 더 빨리 프로세스가
종료된다
int
sys_settickets(void)
{
                int tickets, end_ticks;
                struct proc *p = myproc();

                // 사용자 영역에서 tickets, end_ticks에 해당하는 정수 인자 2개를 읽어온다
                if(argint(0, &tickets) < 0)
                                return -1;
                if(argint(1, &end_ticks) < 0)
                                return -1;

                // 유효성 검사(명세 참고)
                // tickets: 1 이상 && STRIDE_MAX 미만
                if(tickets < 1 || tickets >= STRIDE_MAX)
                                return -1;

                // Stride 스케쥴러 관련 필드 갱신
                p->tickets = tickets;
                p->stride = STRIDE_MAX / tickets;    // 정수 나눗셈(나머지 버림)

                // 테스트 가이드라인 결과와 같도록, 초기 pass값을 0으로 설정
                p->pass = 0;

                // end_ticks: 1미만이면 무시(기존 값 유지), 1 이상이면 지정
                if(end_ticks >= 1)
                                p->end_ticks = end_ticks;

                return 0;
}
```

[trap.c]

```
#include "types.h"
#include "defs.h"
```

```
#include "param.h"
#include "memlayout.h"
#include "mmu.h"
#include "proc.h"
#include "x86.h"
#include "traps.h"
#include "spinlock.h"

// Interrupt descriptor table (shared by all CPUs).
struct gatedesc idt[256];
extern uint vectors[];  // in vectors.S: array of 256 entry pointers
struct spinlock tickslock;
uint ticks;

void
tvinit(void)
{
  int i;

  for(i = 0; i < 256; i++)
    SETGATE(idt[i], 0, SEG_KCODE<<3, vectors[i], 0);
  SETGATE(idt[T_SYSCALL], 1, SEG_KCODE<<3, vectors[T_SYSCALL], DPL_USER);

  initlock(&tickslock, "time");
}

void
idtinit(void)
{
  lidt(idt, sizeof(idt));
}

//PAGEBREAK: 41
void
trap(struct trapframe *tf)
{
  if(tf->trapno == T_SYSCALL){
    if(myproc()->killed)
      exit();
    myproc()->tf = tf;
    syscall();
    if(myproc()->killed)
      exit();
    return;
  }

  switch(tf->trapno){
  case T_IRQ0 + IRQ_TIMER:
    if(cpuid() == 0){
      acquire(&tickslock);
      ticks++;
      wakeup(&ticks);
      release(&tickslock);
    }
    lapiceoi();
    break;
  case T_IRQ0 + IRQ_IDE:
    ideintr();
```

```
    lapiceoi();
    break;
  case T_IRQ0 + IRQ_IDE+1:
    // Bochs generates spurious IDE1 interrupts.
    break;
  case T_IRQ0 + IRQ_KBD:
    kbdintr();
    lapiceoi();
    break;
  case T_IRQ0 + IRQ_COM1:
    uartintr();
    lapiceoi();
    break;
  case T_IRQ0 + 7:
  case T_IRQ0 + IRQ_SPURIOUS:
    cprintf("cpu%d: spurious interrupt at %x:%x\n",
            cpuid(), tf->cs, tf->eip);
    lapiceoi();
    break;

  //PAGEBREAK: 13
  default:
    if(myproc() == 0 || (tf->cs&3) == 0){
      // In kernel, it must be our mistake.
      cprintf("unexpected trap %d from cpu %d eip %x (cr2=0x%x)\n",
              tf->trapno, cpuid(), tf->eip, rcr2());
      panic("trap");
    }
    // In user space, assume process misbehaved.
    cprintf("pid %d %s: trap %d err %d on cpu %d "
            "eip 0x%x addr 0x%x--kill proc\n",
            myproc()->pid, myproc()->name, tf->trapno,
            tf->err, cpuid(), tf->eip, rcr2());
    myproc()->killed = 1;
  }

  // Force process exit if it has been killed and is in user space.
  // (If it is still executing in the kernel, let it keep running
  // until it gets to the regular system call return.)
  if(myproc() && myproc()->killed && (tf->cs&3) == DPL_USER)
    exit();

  // Force process to give up CPU on clock tick.
  // If interrupts were on while locks held, would need to check nlock.
  // myproc(): 현재 실행 중인 내 프로세스를 가리키는 주소를 가져오는 함수
  // 현재 실행 중인 프로세스가 RUNNING 상태이면서 타이머 인터럽트가 발생하면 아래 분기 실행
  if(myproc() && myproc()->state == RUNNING && tf->trapno == T_IRQ0+IRQ_TIMER){
                    struct proc *p = myproc();

                    // 유저 모드에서 타이머에 의해 선점된 경우만 카운트/로그
          if((tf->cs&3) == DPL_USER){

                          // 현재 틱이 종료 틱 수보다 커지면, 로그 한 번 출력하고 exit() 체크
                    if(p->end_ticks > 0 && p->ticks >= p->end_ticks){
                                    if(p->pid > 2 && p->parent && p->parent->pid > 2){
                                   // 커널/초기 프로세스는 로그에서 제외
                                    cprintf("Process %d selected, stride : %d, ticket : %d, pass : %d -> %d
(%d/%d)\n",
```

```
                                                                    p->pid, p->stride, p->tickets, p->pass,
p->pass + p->stride, p->ticks, p->end_ticks);
                        }
                        exit();
                        }

                        // 디버그 로그 (부모/자식 필터)
                        if(p->pid > 2 && p->parent && p->parent->pid > 2){
                                cprintf("Process %d selected, stride : %d, ticket : %d, pass : %d -> %d
(%d/%d)\n",
                                                                    p->pid, p->stride, p->tickets, p->pass,
p->pass + p->stride, p->ticks, p->end_ticks);
                        }
                    }

    // CPU를 양보 -> 스케줄러가 다른 프로세스를 고르게 한다.
    yield();
  }

  // Check if the process has been killed since we yielded
  // yield 이후에 해당 프로세스가 killed 상태라면 exit()
  if(myproc() && myproc()->killed && (tf->cs&3) == DPL_USER)
    exit();
}
```

[user.h]

```
struct stat;
struct rtcdate;

// system calls
int fork(void);
int exit(void) __attribute__((noreturn));
int wait(void);
int pipe(int*);
int write(int, const void*, int);
int read(int, void*, int);
int close(int);
int kill(int);
int exec(char*, char**);
int open(const char*, int);
int mknod(const char*, short, short);
int unlink(const char*);
int fstat(int fd, struct stat*);
int link(const char*, const char*);
int mkdir(const char*);
int chdir(const char*);
int dup(int);
int getpid(void);
char* sbrk(int);
int sleep(int);
int uptime(void);
int settickets(int tickets, int end_ticks);    //시스템콜 추가

// ulib.c
int stat(const char*, struct stat*);
char* strcpy(char*, const char*);
void *memmove(void*, const void*, int);
char* strchr(const char*, char c);
```

```
int strcmp(const char*, const char*);
void printf(int, const char*, ...);
char* gets(char*, int max);
uint strlen(const char*);
void* memset(void*, int, uint);
void* malloc(uint);
void free(void*);
int atoi(const char*);
```

[usys.S]

```
#include "syscall.h"
#include "traps.h"

#define SYSCALL(name) \
  .globl name; \
  name: \
    movl $SYS_ ## name, %eax; \
    int $T_SYSCALL; \
    ret

SYSCALL(fork)
SYSCALL(exit)
SYSCALL(wait)
SYSCALL(pipe)
SYSCALL(read)
SYSCALL(write)
SYSCALL(close)
SYSCALL(kill)
SYSCALL(exec)
SYSCALL(open)
SYSCALL(mknod)
SYSCALL(unlink)
SYSCALL(fstat)
SYSCALL(link)
SYSCALL(mkdir)
SYSCALL(chdir)
SYSCALL(dup)
SYSCALL(getpid)
SYSCALL(sbrk)
SYSCALL(sleep)
SYSCALL(uptime)
SYSCALL(settickets)
```